My Project

Generated by Doxygen 1.16.1

1 Class Index 1
1.1 Class List . . . 1

2 File Index 3
2.1 File List . . . 3

3 Class Documentation 5
3.1 ConfigurationData Struct Reference . . . 5
3.1.1 Detailed Description . . . 6
3.1.2 Member Data Documentation . . . 6
3.1.2.1 humidityMin . . . 6
3.1.2.2 mqttPort . . . 6
3.1.2.3 mqttServer . . . 6
3.1.2.4 valid . . . 7
3.1.2.5 wifiPass . . . 7
3.2 EnvironmentData Struct Reference . . . 7
3.2.1 Detailed Description . . . 8
3.3 ObjectStorage Struct Reference . . . 8
3.3.1 Detailed Description . . . 8

4 File Documentation 9
4.1 microcontrolador/button.cpp File Reference . . . 9
4.1.1 Detailed Description . . . 10
4.1.2 Function Documentation . . . 11
4.1.2.1 buttonInit() . . . 11
4.1.2.2 checkButtonPressed() . . . 11
4.1.2.3 disableButtonInterruptChange() . . . 11
4.1.2.4 disableButtonInterruptRising() . . . 12
4.1.2.5 enableButtonInterruptChange() . . . 12
4.1.2.6 enableButtonInterruptRising() . . . 12
4.1.2.7 longPressButtonISR() . . . 13
4.1.2.8 muteButtonISR() . . . 13
4.1.2.9 resetButtonState() . . . 13
4.1.2.10 wasButtonLongPressed() . . . 14
4.1.2.11 wasButtonShortPress() . . . 14
4.1.2.12 wasMuted() . . . 14
4.1.3 Variable Documentation . . . 15
4.1.3.1 buttonPressStartTime . . . 15
4.1.3.2 flagLongPress . . . 15
4.1.3.3 flagMutePressed . . . 15
4.1.3.4 flagShortPress . . . 15
4.1.3.5 isButtonPressed . . . 16
4.1.3.6 isMuted . . . 16

4.2 microcontrolador/button.h File Reference . . . 16
4.2.1 Detailed Description . . . 17
4.2.2 Function Documentation . . . 17
4.2.2.1 buttonInit() . . . 17
4.2.2.2 disableButtonInterruptChange() . . . 18
4.2.2.3 disableButtonInterruptRising() . . . 18
4.2.2.4 enableButtonInterruptChange() . . . 19
4.2.2.5 enableButtonInterruptRising() . . . 19
4.2.2.6 resetButtonState() . . . 20
4.2.2.7 wasButtonLongPressed() . . . 20
4.2.2.8 wasButtonShortPress() . . . 21
4.2.2.9 wasMuted() . . . 21
4.3 button.h . . . 22
4.4 microcontrolador/buzzer.cpp File Reference . . . 22
4.4.1 Detailed Description . . . 23
4.4.2 Function Documentation . . . 23
4.4.2.1 buzzerInit() . . . 23
4.4.2.2 disableSoundAlert() . . . 23
4.4.2.3 toggleBuzzer() . . . 24
4.4.3 Variable Documentation . . . 24
4.4.3.1 lastBuzzerPlayTime . . . 24
4.5 microcontrolador/buzzer.h File Reference . . . 24
4.5.1 Detailed Description . . . 25
4.5.2 Function Documentation . . . 25
4.5.2.1 buzzerInit() . . . 25
4.5.2.2 disableSoundAlert() . . . 25
4.5.2.3 toggleBuzzer() . . . 26
4.5.3 Variable Documentation . . . 26
4.5.3.1 BUZZER_INTERVAL_MS . . . 26
4.6 buzzer.h . . . 26
4.7 microcontrolador/config.h File Reference . . . 27
4.7.1 Detailed Description . . . 27
4.7.2 Variable Documentation . . . 28
4.7.2.1 PASSWORD_ACCESS_POINT . . . 28
4.7.2.2 PIN_BUTTON_LONG . . . 28
4.7.2.3 PIN_BUTTON_MUTE . . . 28
4.7.2.4 PIN_BUZZER . . . 28
4.7.2.5 PIN_LED . . . 29
4.7.2.6 SSID_ACCESS_POINT . . . 29
4.8 config.h . . . 29
4.9 microcontrolador/config_storage.cpp File Reference . . . 30
4.9.1 Detailed Description . . . 30

4.9.2 Function Documentation . . . 31
4.9.2.1 hasConfigData() . . . 31
4.9.2.2 initConfigStorage() . . . 31
4.9.2.3 loadConfig() . . . 32
4.9.2.4 saveConfigutionData() . . . 33
4.9.3 Variable Documentation . . . 33
4.9.3.1 cfg . . . 33
4.9.3.2 CONFIG_PATH . . . 34
4.10 microcontrolador/config_storage.h File Reference . . . 34
4.10.1 Detailed Description . . . 35
4.10.2 Function Documentation . . . 35
4.10.2.1 hasConfigData() . . . 35
4.10.2.2 initConfigStorage() . . . 36
4.10.2.3 loadConfig() . . . 36
4.10.2.4 saveConfigutionData() . . . 38
4.10.3 Variable Documentation . . . 39
4.10.3.1 cfg . . . 39
4.11 config_storage.h . . . 39
4.12 microcontrolador/display_LCD-1602_I2C.cpp File Reference . . . 40
4.12.1 Detailed Description . . . 41
4.12.2 Function Documentation . . . 42
4.12.2.1 clearLine() . . . 42
4.12.2.2 handleBacklightLCD() . . . 42
4.12.2.3 lcd1602_init() . . . 42
4.12.2.4 lcd1602_showData() . . . 43
4.12.2.5 show_value() . . . 43
4.12.3 Variable Documentation . . . 43
4.12.3.1 TEMP_UPDATE_THRESHOLD . . . 43
4.13 display_LCD-1602_I2C.h . . . 44
4.14 microcontrolador/led.cpp File Reference . . . 44
4.14.1 Detailed Description . . . 44
4.14.2 Function Documentation . . . 45
4.14.2.1 ledInit() . . . 45
4.14.2.2 turnOffLed() . . . 45
4.14.2.3 turnOnLed() . . . 45
4.14.3 Variable Documentation . . . 46
4.14.3.1 lastTimeLedTurnedOn . . . 46
4.14.3.2 LED_INTERVAL_MS . . . 46
4.15 microcontrolador/led.h File Reference . . . 46
4.15.1 Detailed Description . . . 47
4.15.2 Function Documentation . . . 47
4.15.2.1 ledInit() . . . 47

4.15.2.2 turnOffLed() . . . 48
4.15.2.3 turnOnLed() . . . 48
4.16 led.h . . . 48
4.17 microcontrolador/log.cpp File Reference . . . 48
4.17.1 Detailed Description . . . 49
4.17.2 Function Documentation . . . 49
4.17.2.1 log() . . . 49
4.17.2.2 logInit() . . . 50
4.17.3 Variable Documentation . . . 50
4.17.3.1 CURRENT_LOG_LEVEL . . . 50
4.18 microcontrolador/log.h File Reference . . . 50
4.18.1 Detailed Description . . . 51
4.18.2 Enumeration Type Documentation . . . 51
4.18.2.1 LogLevel . . . 51
4.18.3 Function Documentation . . . 51
4.18.3.1 log() . . . 51
4.18.3.2 logInit() . . . 52
4.18.4 Variable Documentation . . . 52
4.18.4.1 CURRENT_LOG_LEVEL . . . 52
4.19 log.h . . . 53
4.20 microcontrolador/mqtt.cpp File Reference . . . 53
4.20.1 Detailed Description . . . 54
4.20.2 Function Documentation . . . 54
4.20.2.1 client() . . . 54
4.20.2.2 publishAlert() . . . 55
4.20.2.3 publishSensorData() . . . 55
4.20.2.4 resendMqttData() . . . 56
4.20.2.5 resetMQTTClient() . . . 56
4.20.2.6 resetStateSendAlert() . . . 57
4.20.3 Variable Documentation . . . 57
4.20.3.1 espClient . . . 57
4.20.3.2 wasSendAlert . . . 57
4.21 microcontrolador/mqtt.h File Reference . . . 57
4.21.1 Detailed Description . . . 58
4.21.2 Function Documentation . . . 59
4.21.2.1 publishAlert() . . . 59
4.21.2.2 publishSensorData() . . . 59
4.21.2.3 resendMqttData() . . . 60
4.21.2.4 resetStateSendAlert() . . . 61
4.21.3 Variable Documentation . . . 61
4.21.3.1 client . . . 61
4.22 mqtt.h . . . 62

4.23 microcontrolador/sensor.cpp File Reference . . . 62
4.23.1 Detailed Description . . . 63
4.23.2 Function Documentation . . . 64
4.23.2.1 initializeSensor() . . . 64
4.23.2.2 readSensorData() . . . 64
4.23.3 Variable Documentation . . . 64
4.23.3.1 aht . . . 64
4.24 microcontrolador/sensor.h File Reference . . . 65
4.24.1 Detailed Description . . . 65
4.24.2 Function Documentation . . . 65
4.24.2.1 initializeSensor() . . . 65
4.24.2.2 readSensorData() . . . 66
4.24.3 Variable Documentation . . . 66
4.24.3.1 aht . . . 66
4.25 sensor.h . . . 66
4.26 microcontrolador/server.h File Reference . . . 67
4.26.1 Detailed Description . . . 67
4.26.2 Function Documentation . . . 68
4.26.2.1 server_handle_loop() . . . 68
4.26.2.2 server_setup_routes() . . . 68
4.26.2.3 startServer() . . . 69
4.26.2.4 stopServer() . . . 69
4.27 server.h . . . 70
4.28 microcontrolador/storage.cpp File Reference . . . 70
4.28.1 Detailed Description . . . 71
4.28.2 Function Documentation . . . 72
4.28.2.1 deleteOneMessage() . . . 72
4.28.2.2 getObjectStorage() . . . 72
4.28.2.3 hasDataStorage() . . . 73
4.28.2.4 initStorage() . . . 73
4.28.2.5 saveStorage() . . . 74
4.28.2.6 storagePrint() . . . 74
4.29 microcontrolador/storage.h File Reference . . . 74
4.29.1 Detailed Description . . . 75
4.29.2 Function Documentation . . . 77
4.29.2.1 deleteOneMessage() . . . 77
4.29.2.2 getObjectStorage() . . . 77
4.29.2.3 hasDataStorage() . . . 78
4.29.2.4 initStorage() . . . 78
4.29.2.5 saveStorage() . . . 79
4.29.2.6 storagePrint() . . . 79
4.30 storage.h . . . 80

4.31 microcontrolador/wifi.cpp File Reference 80
4.31.1 Detailed Description 81
4.31.2 Function Documentation 81
4.31.2.1 connectWiFi() 81
4.31.2.2 createAccessPoint() 82
4.31.2.3 reconnectWifi() 82
4.31.3 Variable Documentation 82
4.31.3.1 ipBuffer 82
4.32 wifi.h 82

5 Examples **83**
5.1 C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/led.cpp 83
5.2 C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/led.h 84
5.3 C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.cpp 85
5.4 C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.h 88

Index **89**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ConfigurationData
Estrutura que contém todos os parâmetros configuráveis do sistema 5
EnvironmentData
Estrutura que armazena os dados ambientais coletados do sensor 7
ObjectStorage
Estrutura que representa um registro armazenado no sistema 8

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

microcontrolador/button.cpp
Implementação do módulo de controle de botões físicos 9
microcontrolador/button.h
Interface do módulo de controle de botões com suporte a interrupções 16
microcontrolador/buzzer.cpp
Implementação do módulo de controle do buzzer 22
microcontrolador/buzzer.h
Interface do módulo de controle do buzzer 24
microcontrolador/config.h
Definições globais de hardware e parâmetros de configuração do sistema 27
microcontrolador/config_storage.cpp
Implementação do subsistema de armazenamento de configuração 30
microcontrolador/config_storage.h
Interface do subsistema de armazenamento persistente de configuração 34
microcontrolador/display_LCD-1602_I2C.cpp
Implementação do controle do display LCD 1602 via I2C 40
microcontrolador/display_LCD-1602_I2C.h 44
microcontrolador/led.cpp
Implementação das funções de controle do LED indicador do sistema 44
microcontrolador/led.h
Interface do módulo de controle do LED indicador 46
microcontrolador/log.cpp
Implementação de funções de logging para depuração e monitoramento do sistema 48
microcontrolador/log.h
Declarações para o sistema de logging do projeto 50
microcontrolador/mqtt.cpp
Implementação do módulo de comunicação MQTT 53
microcontrolador/mqtt.h
Interface pública do módulo de comunicação MQTT 57
microcontrolador/sensor.cpp
Implementação do módulo de leitura do sensor ambiental AHT10 62
microcontrolador/sensor.h
Declarações relacionadas ao uso do sensor AHT10 (temperatura e umidade) 65
microcontrolador/server.h
Interface do módulo responsável pelo servidor HTTP embarcado no ESP8266 67

microcontrolador/storage.cpp
Implementação do módulo de armazenamento persistente de dados de sensores utilizando LittleFS 70
microcontrolador/storage.h
Interface pública do módulo de armazenamento persistente utilizando LittleFS 74
microcontrolador/wifi.cpp
Implementação do gerenciamento de conectividade Wi-Fi do dispositivo 80
microcontrolador/wifi.h 82

Chapter 3

Class Documentation

3.1 ConfigurationData Struct Reference

Estrutura que contém todos os parâmetros configuráveis do sistema.

```
#include <config_storage.h>
```

Public Attributes

- String mqttServer

  Endereço do broker MQTT.
- String **mqttUser**

  Usuário MQTT.
- String **mqttPass**

  Senha MQTT.
- String **mqttTopicData**

  Tópico MQTT de publicação de dados.
- String **mqttTopicAlert**

  Identificador único do dispositivo.
- String **mqttDeviceId**

  Identificador único do dispositivo.
- int mqttPort

  Porta do servidor MQTT.
- String wifiPass

  Porta do servidor MQTT.
- String **wifiSSID**

  Nome da rede Wi-Fi (SSID).
- float **temperatureMax**

  Temperatura máxima permitida.
- float **temperatureMin**

  Temperatura máxima permitida.
- float **humidityMax**

  Umidade máxima permitida.
- float humidityMin

  Umidade mínima permitida.
- bool valid = false

  Indica se os dados são válidos.

3.1.1 Detailed Description

Estrutura que contém todos os parâmetros configuráveis do sistema.

Esta estrutura representa a configuração completa do dispositivo.

É carregada a partir do armazenamento persistente e utilizada por todos os módulos do sistema.

O campo `valid` indica se a configuração foi carregada corretamente.

Note

Se valid == false, a configuração não deve ser utilizada.

3.1.2 Member Data Documentation

3.1.2.1 humidityMin

```
float ConfigurationData::humidityMin
```

Umidade mínima permitida.

Umidade mínima permitida (%)

3.1.2.2 mqttPort

```
int ConfigurationData::mqttPort
```

Porta do servidor MQTT.

Exemplo:

1883

3.1.2.3 mqttServer

```
String ConfigurationData::mqttServer
```

Endereço do broker MQTT.

Exemplo:

broker.hivemq.com

3.1.2.4 valid

```
bool ConfigurationData::valid = false
```

Indica se os dados são válidos.

true → Configuração carregada corretamente

false → Configuração inválida ou inexistente

Warning

Sempre verificar antes de usar a configuração. Indica se os dados carregados são válidos

3.1.2.5 wifiPass

```
String ConfigurationData::wifiPass
```

Porta do servidor MQTT.

Exemplo:

1883

The documentation for this struct was generated from the following file:

- microcontrolador/config_storage.h

3.2 EnvironmentData Struct Reference

Estrutura que armazena os dados ambientais coletados do sensor.

```
#include <sensor.h>
```

Public Attributes

- float **temperature**

  Temperatura em graus Celsius.
- float **humidity**

  Umidade relativa em porcentagem (%).
- bool **valid**

  Indica se os dados lidos são válidos.

3.2.1 Detailed Description

Estrutura que armazena os dados ambientais coletados do sensor.

Contém:

- `temperature`: valor da temperatura em graus Celsius.
- `humidity`: valor da umidade relativa em porcentagem (%).
- `valid`: flag que indica se os dados são válidos (`true`) ou inválidos (`false`).

The documentation for this struct was generated from the following file:

- microcontrolador/sensor.h

3.3 ObjectStorage Struct Reference

Estrutura que representa um registro armazenado no sistema.

```
#include <storage.h>
```

Public Attributes

- float **temperature** = 0.0f

  Temperatura registrada em ℃.
- float **humidity** = 0.0f

  Umidade relativa registrada em %.
- bool **valid** = false

  Indica se o objeto esta vazio.

3.3.1 Detailed Description

Estrutura que representa um registro armazenado no sistema.

Cada objeto contém informações de temperatura, umidade e um indicador de validade.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/↩ microcontrolador/mqtt.cpp.

The documentation for this struct was generated from the following file:

- microcontrolador/storage.h

Chapter 4

File Documentation

4.1 microcontrolador/button.cpp File Reference

Implementação do módulo de controle de botões físicos.

```
#include "c_types.h"
#include "button.h"
#include "log.h"
#include "config.h"
```

Functions

- void IRAM_ATTR muteButtonISR ()

  Rotina de interrupção do botão de mute.
- void IRAM_ATTR longPressButtonISR ()

  Rotina de interrupção para detecção de long press.
- void buttonInit (uint8_t pin)

  Inicializa o pino do botão.
- void enableButtonInterruptRising (uint8_t pin)

  Habilita interrupção por borda de subida.
- void disableButtonInterruptRising (uint8_t pin)

  Desabilita interrupção por borda de subida.
- void enableButtonInterruptChange (uint8_t pin)

  Habilita interrupção por mudança de estado.
- void disableButtonInterruptChange (uint8_t pin)

  Desabilita interrupção por mudança de estado.
- bool checkButtonPressed (volatile bool *buttonPressed)

  Verifica se o botão foi pressionado.
- bool wasMuted ()

  Retorna o estado atual do modo mute.
- bool wasButtonLongPressed ()

  Verifica ocorrência de long press.
- bool wasButtonShortPress ()

  Verifica ocorrência de short press.
- void resetButtonState ()

  Reseta o estado do botão mute.

Variables

- volatile unsigned long buttonPressStartTime = 0

  Armazena o instante em que o botão foi pressionado.
- volatile bool flagLongPress = false

  Flag que indica ocorrência de long press.
- volatile bool flagShortPress = false

  Flag que indica ocorrência de short press.
- volatile bool isButtonPressed = false

  Indica se o botão está atualmente pressionado.
- volatile bool flagMutePressed = false

  Flag de pressionamento do botão mute.
- bool **isLCDturnOn** = false
- bool isMuted = false

  Indica se o sistema está em modo mute.

4.1.1 Detailed Description

Implementação do módulo de controle de botões físicos.

Este módulo é responsável pelo gerenciamento de dois botões físicos utilizados no sistema:

- Botão de mute (pressionamento curto)
- Botão de long press (pressionamento longo)

Funcionalidades implementadas:

- Detecção de pressionamento curto
- Detecção de pressionamento longo
- Alternância de estado lógico (toggle)
- Tratamento de debounce por software
- Uso de interrupções externas (ISR)

O módulo utiliza variáveis do tipo volatile para garantir integridade no acesso concorrente entre o código principal e as rotinas de interrupção.

Note

Este módulo depende das definições de pinos em config.h

Date

2026

4.1.2 Function Documentation

4.1.2.1 buttonInit()

```
void buttonInit (
            uint8_t pin)
```

Inicializa o pino do botão.

Parameters

pin	Número do pino digital conectado ao botão.

Configura o pino como entrada com resistor pull-up interno.

Note

Deve ser chamada durante o setup do sistema.

4.1.2.2 checkButtonPressed()

```
bool checkButtonPressed (
            volatile bool * buttonPressed)
```

Verifica se o botão foi pressionado.

Se a flag de clique estiver ativa, ela é limpa e retorna `true`.

Parameters

buttonPressed	Ponteiro para a flag de estado do botão.

Returns

`true` se o botão foi pressionado.

`false` caso contrário.

4.1.2.3 disableButtonInterruptChange()

```
void disableButtonInterruptChange (
            uint8_t pin)
```

Desabilita interrupção por mudança de estado.

Desabilita a interrupção do botão de pressão longa e curta.

Parameters

pin	Pino do botão.

Utilizada para detectar início e fim do pressionamento, permitindo calcular sua duração.

4.1.2.4 disableButtonInterruptRising()

```
void disableButtonInterruptRising (
            uint8_t pin)
```

Desabilita interrupção por borda de subida.

Desabilita a interrupção do botão de mute.

Parameters

pin	Pino do botão.

Associa o pino à ISR responsável pelo tratamento de pressionamento do botão mute.

4.1.2.5 enableButtonInterruptChange()

```
void enableButtonInterruptChange (
            uint8_t pin)
```

Habilita interrupção por mudança de estado.

Habilita a interrupção para detecção de pressão longa e curta (botao configuração/turn on).

Parameters

pin	Pino do botão.

Utilizada para detectar início e fim do pressionamento, permitindo calcular sua duração.

4.1.2.6 enableButtonInterruptRising()

```
void enableButtonInterruptRising (
            uint8_t pin)
```

Habilita interrupção por borda de subida.

Habilita a interrupção para detecção de clique curto (mute).

Parameters

pin	Pino do botão.

Associa o pino à ISR responsável pelo tratamento de pressionamento do botão mute.

4.1.2.7 longPressButtonISR()

```
void IRAM_ATTR longPressButtonISR ()
```

Rotina de interrupção para detecção de long press.

Monitora o tempo em que o botão permanece pressionado.

Comportamento:

- Se pressionado por mais de 15 segundos → long press
- Se pressionado entre 100 ms e 15 segundos → short press

Note

Executada em contexto de interrupção.

4.1.2.8 muteButtonISR()

```
void IRAM_ATTR muteButtonISR ()
```

Rotina de interrupção do botão de mute.

Executada automaticamente quando ocorre uma borda de subida no pino configurado.

Implementa debounce por software para evitar múltiplas detecções causadas por ruído mecânico.

Note

Esta função é executada em contexto de interrupção.

< Marca o instante da última interrupção

4.1.2.9 resetButtonState()

```
void resetButtonState ()
```

Reseta o estado do botão mute.

Reseta completamente o estado interno do botão.

Define o sistema como não mutado.

Note

Usado em reinicializações ou mudança de modo.

4.1.2.10 wasButtonLongPressed()

```
bool wasButtonLongPressed ()
```

Verifica ocorrência de long press.

Verifica se ocorreu um evento de pressão longa.

Returns

true Se ocorreu long press.

false Caso contrário.

4.1.2.11 wasButtonShortPress()

```
bool wasButtonShortPress ()
```

Verifica ocorrência de short press.

Verifica se ocorreu um evento de clique curto no botao de configuracao/ligar lcd.

Returns

true Se ocorreu short press.

false Caso contrário.

Note

A flag é automaticamente resetada após leitura.

4.1.2.12 wasMuted()

```
bool wasMuted ()
```

Retorna o estado atual do modo mute.

Verifica se o sistema entrou no estado de mute.

Returns

true Sistema está mutado.

false Sistema não está mutado.

Alterna o estado interno quando um novo pressionamento é detectado.

4.1.3 Variable Documentation

4.1.3.1 buttonPressStartTime

```
volatile unsigned long buttonPressStartTime = 0
```

Armazena o instante em que o botão foi pressionado.

Utilizado para calcular o tempo de duração do pressionamento e determinar se foi um short press ou long press.

Note

Variável modificada dentro de ISR.

4.1.3.2 flagLongPress

```
volatile bool flagLongPress = false
```

Flag que indica ocorrência de long press.

É ativada quando o botão permanece pressionado por tempo superior ao limite definido.

Note

Acessada no loop principal.

4.1.3.3 flagMutePressed

```
volatile bool flagMutePressed = false
```

Flag de pressionamento do botão mute.

Utilizada para sinalizar que ocorreu um clique válido, após tratamento de debounce.

4.1.3.4 flagShortPress

```
volatile bool flagShortPress = false
```

Flag que indica ocorrência de short press.

Note

Definida dentro da ISR e consumida no loop principal.

4.1.3.5 isButtonPressed

```
volatile bool isButtonPressed = false
```

Indica se o botão está atualmente pressionado.

Utilizado para controlar a transição de estados impedindo a atualizacao do buttonPressStartTime para estado atual e obtendo o momento em que o botao foi precionado.

4.1.3.6 isMuted

```
bool isMuted = false
```

Indica se o sistema está em modo mute.

Alterna seu valor a cada clique válido no botão mute.

4.2 microcontrolador/button.h File Reference

Interface do módulo de controle de botões com suporte a interrupções.

```
#include <Arduino.h>
```

Functions

- void buttonInit (uint8_t pin)

  Inicializa o pino do botão.
- void enableButtonInterruptRising (uint8_t pin)

  Habilita a interrupção para detecção de clique curto (mute).
- void disableButtonInterruptRising (uint8_t pin)

  Desabilita a interrupção do botão de mute.
- void enableButtonInterruptChange (uint8_t pin)

  Habilita a interrupção para detecção de pressão longa e curta (botao configuração/turn on).
- void disableButtonInterruptChange (uint8_t pin)

  Desabilita a interrupção do botão de pressão longa e curta.
- bool wasMuted ()

  Verifica se o sistema entrou no estado de mute.
- bool wasButtonLongPressed ()

  Verifica se ocorreu um evento de pressão longa.
- bool wasButtonShortPress ()

  Verifica se ocorreu um evento de clique curto no botao de configuracao/ligar lcd.
- void resetButtonState ()

  Reseta completamente o estado interno do botão.

4.2.1 Detailed Description

Interface do módulo de controle de botões com suporte a interrupções.

Este módulo fornece uma abstração para o gerenciamento de botões físicos utilizando interrupções de hardware, permitindo a detecção eficiente dos seguintes eventos:

- Clique curto (toggle) — utilizado para alternar o estado de mute
- Pressão longa (long press e short press) — utilizado para modo configuração e mostrar dados no display lcd

O módulo implementa internamente:

- Tratamento de debounce
- Controle de flags de eventos
- Lógica de alternância de estado (toggle)

A implementação está localizada em button.cpp.

4.2.2 Function Documentation

4.2.2.1 buttonInit()

```
void buttonInit (
            uint8_t pin)
```

Inicializa o pino do botão.

Configura o pino especificado como entrada digital com resistor de pull-up interno ativado.

Essa configuração é recomendada para botões conectados entre o pino e o GND, evitando estados flutuantes.

Esta função deve ser chamada durante a inicialização do sistema (ex.: setup()) antes de habilitar interrupções.

Parameters

pin	Número do pino digital onde o botão está conectado.
pin	Número do pino digital conectado ao botão.

Configura o pino como entrada com resistor pull-up interno.

Note

Deve ser chamada durante o setup do sistema.

4.2.2.2 disableButtonInterruptChange()

```
void disableButtonInterruptChange (
            uint8_t pin)
```

Desabilita a interrupção do botão de pressão longa e curta.

Após esta chamada, eventos de pressão longa não serão mais detectados.

Parameters

pin	Número do pino do botão.

Desabilita a interrupção do botão de pressão longa e curta.

Parameters

pin	Pino do botão.

Utilizada para detectar início e fim do pressionamento, permitindo calcular sua duração.

4.2.2.3 disableButtonInterruptRising()

```
void disableButtonInterruptRising (
            uint8_t pin)
```

Desabilita a interrupção do botão de mute.

Após esta chamada, eventos de clique curto não serão mais detectados até que a interrupção seja habilitada novamente.

Parameters

pin	Número do pino do botão.

Desabilita a interrupção do botão de mute.

Parameters

pin	Pino do botão.

Associa o pino à ISR responsável pelo tratamento de pressionamento do botão mute.

4.2.2.4 enableButtonInterruptChange()

```
void enableButtonInterruptChange (
            uint8_t pin)
```

Habilita a interrupção para detecção de pressão longa e curta (botao configuração/turn on).

Configura uma interrupção para o evento de mudança de estado (CHANGE), permitindo medir o tempo em que o botão permanece pressionado.

Essa interrupção é utilizada internamente para detectar pressões longas.

Parameters

pin	Número do pino do botão.

Habilita a interrupção para detecção de pressão longa e curta (botao configuração/turn on).

Parameters

pin	Pino do botão.

Utilizada para detectar início e fim do pressionamento, permitindo calcular sua duração.

4.2.2.5 enableButtonInterruptRising()

```
void enableButtonInterruptRising (
            uint8_t pin)
```

Habilita a interrupção para detecção de clique curto (mute).

Configura uma interrupção para o evento de borda de subida (RISING), indicando que o botão foi pressionado.

Quando o evento ocorre, a rotina de interrupção associada é executada, atualizando o estado interno do módulo.

Parameters

pin	Número do pino do botão.

Habilita a interrupção para detecção de clique curto (mute).

Parameters

pin	Pino do botão.

Associa o pino à ISR responsável pelo tratamento de pressionamento do botão mute.

4.2.2.6 resetButtonState()

```
void resetButtonState ()
```

Reseta completamente o estado interno do botão.

Define o sistema como não mutado e limpa flags internas associadas.

Deve ser utilizada em:

- Inicialização do sistema
- Reinicializações
- Reset manual do estado lógico

Reseta completamente o estado interno do botão.

Define o sistema como não mutado.

Note

Usado em reinicializações ou mudança de modo.

4.2.2.7 wasButtonLongPressed()

```
bool wasButtonLongPressed ()
```

Verifica se ocorreu um evento de pressão longa.

Retorna true apenas uma vez para cada evento detectado.

Após a leitura, o estado interno é mantido ou redefinido conforme a implementação no arquivo button.cpp.

Returns

true Pressão longa detectada.
false Nenhuma pressão longa detectada.

Verifica se ocorreu um evento de pressão longa.

Returns

true Se ocorreu long press.
false Caso contrário.

4.2.2.8 wasButtonShortPress()

```
bool wasButtonShortPress ()
```

Verifica se ocorreu um evento de clique curto no botao de configuracao/ligar lcd.

Returns

true Clique curto detectado.

false Nenhum evento detectado.

Verifica se ocorreu um evento de clique curto no botao de configuracao/ligar lcd.

Returns

true Se ocorreu short press.

false Caso contrário.

Note

A flag é automaticamente resetada após leitura.

4.2.2.9 wasMuted()

```
bool wasMuted ()
```

Verifica se o sistema entrou no estado de mute.

Essa função verifica se ocorreu um evento de clique curto e alterna o estado interno de mute.

Returns

true Sistema está em mute.

false Sistema está ativo.

Verifica se o sistema entrou no estado de mute.

Returns

true Sistema está mutado.

false Sistema não está mutado.

Alterna o estado interno quando um novo pressionamento é detectado.

4.3 button.h

Go to the documentation of this file.

```
00001
00020
00021 #ifndef BUTTON_H
00022 #define BUTTON_H
00023
00024 #include <Arduino.h>
00025
00026
00027 // ==============================================================================
00028 // Inicialização
00029 // ==============================================================================
00030
00045 void buttonInit(uint8_t pin);
00046
00047
00048 // ==============================================================================
00049 // Controle de interrupções -- clique curto
00050 // ==============================================================================
00051
00063 void enableButtonInterruptRising(uint8_t pin);
00064
00065
00074 void disableButtonInterruptRising(uint8_t pin);
00075
00076 // ==============================================================================
00077 // Controle de interrupções -- pressão longa e pressão curta
00078 // ==============================================================================
00079
00090 void enableButtonInterruptChange(uint8_t pin);
00091
00099 void disableButtonInterruptChange(uint8_t pin);
00100
00101 // ==============================================================================
00102 // Consulta de estado
00103 // ==============================================================================
00104
00114 bool wasMuted();
00115
00127 bool wasButtonLongPressed();
00128
00136 bool wasButtonShortPress();
00137
00138
00139 // ==============================================================================
00140 // Controle de estado
00141 // ==============================================================================
00142
00154 void resetButtonState();
00155
00156
00157 #endif
```

4.4 microcontrolador/buzzer.cpp File Reference

Implementação do módulo de controle do buzzer.

```
#include "buzzer.h"
#include "log.h"
#include "config.h"
```

Functions

- void buzzerInit ()

  Inicializa o buzzer.
- void toggleBuzzer (unsigned long now)

  Alterna o estado do buzzer.
- void disableSoundAlert ()

  Desativa o alerta sonoro.

Variables

- unsigned long lastBuzzerPlayTime = 0

  Armazena o tempo da última alternância do buzzer.

4.4.1 Detailed Description

Implementação do módulo de controle do buzzer.

Este módulo é responsável por controlar o buzzer utilizado para alertas sonoros no sistema.

O buzzer opera em modo intermitente, alternando entre ligado e desligado em um intervalo de tempo definido.

Funcionalidades:

- Inicialização do pino
- Geração de alerta sonoro intermitente
- Desativação do alerta sonoro

Este módulo é utilizado pelo sistema de monitoramento ambiental para sinalizar condições de erro.

Note

O pino (PIN_BUZZER) e o intervalo (BUZZER_INTERVAL_MS) são definidos em config.h

Date

2026

4.4.2 Function Documentation

4.4.2.1 buzzerInit()

```
void buzzerInit ()
```

Inicializa o buzzer.

Inicializa o buzzer configurando o pino como saída.

Configura o pino do buzzer como saída digital e garante que o dispositivo inicie desligado.

Esta função deve ser chamada durante a inicialização do sistema.

4.4.2.2 disableSoundAlert()

```
void disableSoundAlert ()
```

Desativa o alerta sonoro.

Desliga o buzzer forçando nível lógico baixo no pino.

Desliga o buzzer imediatamente, colocando o pino em nível lógico baixo.

4.4.2.3 toggleBuzzer()

```
void toggleBuzzer (
            unsigned long now)
```

Alterna o estado do buzzer.

Alterna o estado do buzzer respeitando o intervalo configurado.

Parameters

now	Tempo atual em milissegundos (millis())

Alterna entre ligado e desligado em intervalos definidos, criando um efeito sonoro intermitente.

Essa abordagem evita o uso de delay(), permitindo que o sistema continue executando outras tarefas.

4.4.3 Variable Documentation

4.4.3.1 lastBuzzerPlayTime

```
unsigned long lastBuzzerPlayTime = 0
```

Armazena o tempo da última alternância do buzzer.

Utilizada para controlar o intervalo entre ativações, garantindo que o buzzer opere de forma intermitente sem bloquear o fluxo principal do programa.

Note

Variável de uso interno do módulo.

4.5 microcontrolador/buzzer.h File Reference

Interface do módulo de controle do buzzer.

```
#include <Arduino.h>
```

Functions

- void buzzerInit ()

  Inicializa o buzzer configurando o pino como saída.
- void toggleBuzzer (unsigned long now)

  Alterna o estado do buzzer respeitando o intervalo configurado.
- void disableSoundAlert ()

  Desliga o buzzer forçando nível lógico baixo no pino.

Variables

- constexpr unsigned long BUZZER_INTERVAL_MS = 2000

  Intervalo de alternância do buzzer em milissegundos.

4.5.1 Detailed Description

Interface do módulo de controle do buzzer.

Este módulo define a interface para controle do buzzer, incluindo inicialização, alternância de estado e desativação do alerta sonoro. O buzzer é utilizado para indicar condições anormais detectadas pelo sistema de monitoramento.

Date

2026

4.5.2 Function Documentation

4.5.2.1 buzzerInit()

```
void buzzerInit ()
```

Inicializa o buzzer configurando o pino como saída.

Deve ser chamada no setup() do programa antes de qualquer uso do buzzer.

Inicializa o buzzer configurando o pino como saída.

Configura o pino do buzzer como saída digital e garante que o dispositivo inicie desligado.

Esta função deve ser chamada durante a inicialização do sistema.

4.5.2.2 disableSoundAlert()

```
void disableSoundAlert ()
```

Desliga o buzzer forçando nível lógico baixo no pino.

Deve ser chamada quando as condições de alerta não estão mais presentes.

Desliga o buzzer forçando nível lógico baixo no pino.

Desliga o buzzer imediatamente, colocando o pino em nível lógico baixo.

4.5.2.3 toggleBuzzer()

```
void toggleBuzzer (
            unsigned long now)
```

Alterna o estado do buzzer respeitando o intervalo configurado.

Parameters

now	Tempo atual em milissegundos.

Note

Deve ser chamada periodicamente no loop principal.

Alterna o estado do buzzer respeitando o intervalo configurado.

Parameters

now	Tempo atual em milissegundos (millis())

Alterna entre ligado e desligado em intervalos definidos, criando um efeito sonoro intermitente.

Essa abordagem evita o uso de delay(), permitindo que o sistema continue executando outras tarefas.

4.5.3 Variable Documentation

4.5.3.1 BUZZER_INTERVAL_MS

```
unsigned long BUZZER_INTERVAL_MS = 2000  [constexpr]
```

Intervalo de alternância do buzzer em milissegundos.

Define o período do alerta sonoro intermitente.

4.6 buzzer.h

Go to the documentation of this file.

```
00001
00011
00012
00013 #ifndef BUZZER_H
00014 #define BUZZER_H
00015 #include <Arduino.h>
00016
00017
00018
00025 constexpr unsigned long BUZZER_INTERVAL_MS = 2000;
00026
00032 void buzzerInit();
00033
00041 void toggleBuzzer(unsigned long now);
00042
00048 void disableSoundAlert();
00049
00050 #endif
```

4.7 microcontrolador/config.h File Reference

Definições globais de hardware e parâmetros de configuração do sistema.

```
#include <stdint.h>
```

Variables

- constexpr uint8_t PIN_BUTTON_MUTE = 13

  Pino do botão de mute.
- constexpr uint8_t PIN_BUTTON_LONG = 2

  Pino do botão de pressão longa.
- constexpr uint8_t **PIN_SENSOR_SCL** = 5

  Pino SCL do sensor AHT10 (I2C Clock).
- constexpr uint8_t **PIN_SENSOR_SDA** = 4

  Pino SDA do sensor AHT10 (I2C Data).
- constexpr uint8_t PIN_BUZZER = 12

  Pino de controle do buzzer.
- constexpr uint8_t PIN_LED = 0

  Pino do LED de status.
- constexpr const char ∗ SSID_ACCESS_POINT = "xxxxxxxxxxxxx"

  Nome da rede Wi-Fi criada pelo dispositivo (Access Point).
- constexpr const char ∗ PASSWORD_ACCESS_POINT = "xxxxxxxxxxxx"

  Senha da rede Wi-Fi criada pelo dispositivo.

4.7.1 Detailed Description

Definições globais de hardware e parâmetros de configuração do sistema.

Este arquivo centraliza todas as constantes utilizadas para:

- Mapeamento de pinos do microcontrolador
- Configuração do modo Access Point (AP)

Isso facilita manutenção, portabilidade e organização do firmware.

Note

Sempre que houver alteração de hardware, este arquivo deve ser atualizado.

Warning

Evite modificar esses valores em tempo de execução. Eles representam a configuração física do dispositivo.

4.7.2 Variable Documentation

4.7.2.1 PASSWORD_ACCESS_POINT

```
const char* PASSWORD_ACCESS_POINT = "xxxxxxxxxxxx"  [constexpr]
```

Senha da rede Wi-Fi criada pelo dispositivo.

Warning

Recomenda-se utilizar senha forte em ambiente de produção.

4.7.2.2 PIN_BUTTON_LONG

```
uint8_t PIN_BUTTON_LONG = 2  [constexpr]
```

Pino do botão de pressão longa.

Utilizado para comandos especiais, como:

- Reset
- Modo configuração

4.7.2.3 PIN_BUTTON_MUTE

```
uint8_t PIN_BUTTON_MUTE = 13  [constexpr]
```

Pino do botão de mute.

Utilizado para ativar/desativar o som do sistema.

Evento detectado via interrupção.

4.7.2.4 PIN_BUZZER

```
uint8_t PIN_BUZZER = 12  [constexpr]
```

Pino de controle do buzzer.

Utilizado para alertas sonoros.

4.7.2.5 PIN_LED

```
uint8_t PIN_LED = 0  [constexpr]
```

Pino do LED de status.

Utilizado para indicar estados do sistema.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/↩ microcontrolador/led.cpp.

4.7.2.6 SSID_ACCESS_POINT

```
const char* SSID_ACCESS_POINT = "xxxxxxxxxxxxx"  [constexpr]
```

Nome da rede Wi-Fi criada pelo dispositivo (Access Point).

Esta rede é utilizada para acessar o portal de configuração do dispositivo.

Exemplo:

Device_Config

4.8 config.h

Go to the documentation of this file.

```
00001
00020
00021 #ifndef CONFIG_H
00022 #define CONFIG_H
00023
00024 #include <stdint.h>
00025
00026
00027
00028 // ============================================================================
00029 // Mapeamento de pinos
00030 // ============================================================================
00031
00039 constexpr uint8_t PIN_BUTTON_MUTE = 13;
00040
00041
00050 constexpr uint8_t PIN_BUTTON_LONG = 2;
00051
00052
00056 constexpr uint8_t PIN_SENSOR_SCL = 5;
00057
00058
00062 constexpr uint8_t PIN_SENSOR_SDA = 4;
00063
00064
00070 constexpr uint8_t PIN_BUZZER = 12;
00071
00072
00078 constexpr uint8_t PIN_LED = 0;
00079
00080
00081
00082 // ============================================================================
00083 // Configuração do Access Point
00084 // ============================================================================
00085
00096 constexpr const char* SSID_ACCESS_POINT = "xxxxxxxxxxxxx";
00097
00098
00099
00106 constexpr const char* PASSWORD_ACCESS_POINT = "xxxxxxxxxxxx";
00107
00108
00109 #endif
```

4.9 microcontrolador/config_storage.cpp File Reference

Implementação do subsistema de armazenamento de configuração.

```
#include "config_storage.h"
#include <ArduinoJson.h>
#include <LittleFS.h>
#include "log.h"
```

Functions

- void initConfigStorage ()

  Inicializa o subsistema de armazenamento de configuração.
- bool hasConfigData ()

  Verifica se existe um arquivo de configuração salvo.
- ConfigurationData loadConfig ()

  Carrega as configurações do sistema a partir do arquivo.
- bool saveConfigutionData (const ConfigurationData &config)

  Salva a configuração no sistema de arquivos.

Variables

- const char ∗ CONFIG_PATH = "/config.json"

  Caminho do arquivo de configuração no LittleFS.
- ConfigurationData cfg

  Estrutura global contendo a configuração ativa do sistema.

4.9.1 Detailed Description

Implementação do subsistema de armazenamento de configuração.

Este módulo é responsável por persistir e recuperar as configurações do sistema utilizando o sistema de arquivos LittleFS em formato JSON.

As configurações incluem:

- Parâmetros de conexão Wi-Fi
- Configuração do broker MQTT
- Tópicos de publicação
- Limites de temperatura e umidade

O armazenamento é realizado no arquivo:

`/config.json`

Este módulo garante:

- Persistência entre reinicializações
- Carregamento seguro
- Validação básica

Note

O sistema de arquivos LittleFS deve ser montado antes da utilização deste módulo.

4.9.2 Function Documentation

4.9.2.1 hasConfigData()

```
bool hasConfigData ()
```

Verifica se existe um arquivo de configuração salvo.

Verifica se existe configuração salva.

Essa função verifica a existência do arquivo `/config.json` no sistema de arquivos LittleFS.

Return values

true	Arquivo existe.
false	Arquivo não existe.

4.9.2.2 initConfigStorage()

```
void initConfigStorage ()
```

Inicializa o subsistema de armazenamento de configuração.

Inicializa o subsistema de configuração.

Esta função prepara o módulo de armazenamento de configuração.

Atualmente, assume que o LittleFS já foi montado por outro módulo (storage.h)

Apenas registra uma mensagem de log indicando que o sistema está pronto.

Note

Esta função não monta o LittleFS.

See also

LittleFS.begin()

4.9.2.3 loadConfig()

```
ConfigurationData loadConfig ()
```

Carrega as configurações do sistema a partir do arquivo.

Carrega as configurações persistentes.

Esta função realiza as seguintes etapas:

1. Abre o arquivo `/config.json`
2. Lê o conteúdo JSON
3. Converte os dados para a estrutura ConfigurationData
4. Retorna a estrutura preenchida

Em caso de erro:

- Arquivo inexistente
- Erro de leitura
- JSON inválido

retorna uma estrutura com:

valid = false

Returns

ConfigurationData Estrutura contendo as configurações carregadas.

Note

O tamanho do JSON é limitado a 768 bytes.

Warning

Se o JSON estiver corrompido, os dados serão descartados.

4.9.2.4 saveConfigutionData()

```
bool saveConfigutionData (
            const ConfigurationData & config)
```

Salva a configuração no sistema de arquivos.

Salva configuração no armazenamento persistente.

Esta função:

- Cria ou sobrescreve o arquivo `/config.json`
- Converte a estrutura ConfigurationData para JSON
- Salva no LittleFS
- Atualiza a variável global cfg

Parameters

config	Configuração a ser salva.

Return values

true	Configuração salva com sucesso.
false	Falha ao salvar.

Note

Após salvar, a configuração é recarregada automaticamente.

Warning

A configuração anterior será sobrescrita.

4.9.3 Variable Documentation

4.9.3.1 cfg

```
ConfigurationData cfg
```

Estrutura global contendo a configuração ativa do sistema.

Configuração global ativa do sistema.

Esta variável é atualizada após o carregamento ou salvamento das configurações.

Warning

Deve ser acessada somente após loadConfig() ou saveConfigutionData().

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/↩ microcontrolador/mqtt.cpp.

4.9.3.2 CONFIG_PATH

```
const char* CONFIG_PATH = "/config.json"
```

Caminho do arquivo de configuração no LittleFS.

Este arquivo armazena todas as configurações do sistema em formato JSON.

4.10 microcontrolador/config_storage.h File Reference

Interface do subsistema de armazenamento persistente de configuração.

```
#include <Arduino.h>
```

Classes

- struct ConfigurationData

  Estrutura que contém todos os parâmetros configuráveis do sistema.

Functions

- void initConfigStorage ()

  Inicializa o subsistema de configuração.
- ConfigurationData loadConfig ()

  Carrega as configurações persistentes.
- bool saveConfigutionData (const ConfigurationData &config)

  Salva configuração no armazenamento persistente.
- bool hasConfigData ()

  Verifica se existe configuração salva.

Variables

- ConfigurationData cfg

  Configuração global ativa do sistema.

4.10.1 Detailed Description

Interface do subsistema de armazenamento persistente de configuração.

Este módulo define a estrutura de dados e as funções responsáveis por gerenciar as configurações persistentes do sistema.

As configurações são armazenadas no sistema de arquivos interno (LittleFS) em formato JSON no arquivo:

`/config.json`

Este subsistema permite:

- Salvar configurações
- Carregar configurações
- Verificar existência de configuração

As configurações incluem:

- Credenciais Wi-Fi
- Configurações MQTT
- Limites operacionais de sensores

Note

O sistema de arquivos LittleFS deve ser inicializado antes do uso.

Warning

Sempre verifique o campo `valid` após loadConfig().

4.10.2 Function Documentation

4.10.2.1 hasConfigData()

```
bool hasConfigData ()
```

Verifica se existe configuração salva.

Return values

true	Existe
false	Não existe

Verifica se existe configuração salva.

Essa função verifica a existência do arquivo `/config.json` no sistema de arquivos LittleFS.

Return values

true	Arquivo existe.
false	Arquivo não existe.

4.10.2.2 initConfigStorage()

```
void initConfigStorage ()
```

Inicializa o subsistema de configuração.

Prepara o módulo para operações de leitura e escrita.

Note

Não monta o LittleFS.

Inicializa o subsistema de configuração.

Esta função prepara o módulo de armazenamento de configuração.

Atualmente, assume que o LittleFS já foi montado por outro módulo (storage.h)

Apenas registra uma mensagem de log indicando que o sistema está pronto.

Note

Esta função não monta o LittleFS.

See also

LittleFS.begin()

4.10.2.3 loadConfig()

```
ConfigurationData loadConfig ()
```

Carrega as configurações persistentes.

Lê o arquivo `/config.json` e converte para ConfigurationData.

Returns

Estrutura contendo a configuração carregada.

Warning

Verifique config.valid antes de usar.

Carrega as configurações persistentes.

Esta função realiza as seguintes etapas:

1. Abre o arquivo `/config.json`
2. Lê o conteúdo JSON
3. Converte os dados para a estrutura ConfigurationData
4. Retorna a estrutura preenchida

Em caso de erro:

- Arquivo inexistente
- Erro de leitura
- JSON inválido

retorna uma estrutura com:

valid = false

Returns

ConfigurationData Estrutura contendo as configurações carregadas.

Note

O tamanho do JSON é limitado a 768 bytes.

Warning

Se o JSON estiver corrompido, os dados serão descartados.

4.10.2.4 saveConfigutionData()

```
bool saveConfigutionData (
            const ConfigurationData & config)
```

Salva configuração no armazenamento persistente.

Parameters

config	Configuração a ser salva.

Return values

true	Sucesso
false	Falha

Salva configuração no armazenamento persistente.

Esta função:

- Cria ou sobrescreve o arquivo `/config.json`
- Converte a estrutura ConfigurationData para JSON
- Salva no LittleFS
- Atualiza a variável global cfg

Parameters

config	Configuração a ser salva.

Return values

true	Configuração salva com sucesso.
false	Falha ao salvar.

Note

Após salvar, a configuração é recarregada automaticamente.

Warning

A configuração anterior será sobrescrita.

4.10.3 Variable Documentation

4.10.3.1 cfg

```
ConfigurationData cfg [extern]
```

Configuração global ativa do sistema.

Esta variável contém a configuração atualmente carregada.

É atualizada automaticamente após:

- loadConfig()
- saveConfigutionData()

Warning

Não usar antes de carregar configuração.

Configuração global ativa do sistema.

Esta variável é atualizada após o carregamento ou salvamento das configurações.

Warning

Deve ser acessada somente após loadConfig() ou saveConfigutionData().

4.11 config_storage.h

Go to the documentation of this file.

```
00001
00030
00031 #ifndef CONFIG_STORAGE
00032 #define CONFIG_STORAGE
00033
00034 #include <Arduino.h>
00035
00050 struct ConfigurationData
00051 {
00052
00060   String mqttServer;
00061
00065   String mqttUser;
00066
00070   String mqttPass;
00071
00075   String mqttTopicData;
00076
00080   String mqttTopicAlert;
00081
00085   String mqttDeviceId;
00086
00094   int mqttPort;
00095
00103   String wifiPass;
00104
00108   String wifiSSID;
00109
00113   float temperatureMax;
00114
00118   float temperatureMin;
00119
```

```
  float humidityMax;

  float humidityMin;

  bool valid = false;
};


// ==========================================================================
// Funções
// ==========================================================================


void initConfigStorage();

ConfigurationData loadConfig();

bool saveConfigutionData(const ConfigurationData &config);

bool hasConfigData();

// ==========================================================================
// Variáveis globais
// ==========================================================================


extern ConfigurationData cfg;

#endif
```

4.12 microcontrolador/display_LCD-1602_I2C.cpp File Reference

Implementação do controle do display LCD 1602 via I2C.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <math.h>
#include "display_LCD-1602_I2C.h"
#include "log.h"
```

Macros

- #define **TEMPERATURE_ALERT_MESSAGE** F("ALERT_T: ")

  Mensagem exibida quando temperatura ultrapassa o limite.
- #define **TEMPERATURE_MESSAGE** F("T: ")

  Mensagem exibida quando temperatura está dentro do limite.
- #define **HUMIDITY_ALERT_MESSAGE** F("ALERT_U: ")

  Mensagem exibida quando umidade ultrapassa o limite.
- #define **HUMIDITY_MESSAGE** F("U: ")

  Mensagem exibida quando umidade está dentro do limite.

Functions

- LiquidCrystal_I2C **lcd** (I2C_ADDR, LCD_COLUMNS, LCD_LINES)

  Instância do LCD.
- void lcd1602_init ()

  Inicializa o display.
- void clearLine (uint8_t line)

  Limpa uma linha do display.

- void show_value (uint8_t line, const __FlashStringHelper *normalMsg, const __FlashStringHelper *alertMsg, float value, bool alert, const char *unit)

  Exibe no display o valor de temperatura ou umidade.
- void lcd1602_showData (float temp, float humi, bool alertTemp, bool alertHumi)

  Exibe os valores de temperatura e umidade no display LCD.
- void **turnOnBacklight** ()

  Liga o backlight.
- void **turnOffBacklight** ()

  Desliga o backlight.
- void handleBacklightLCD (bool isAlertState, bool commandTurnOnLCD)

  Controla o estado do backlight.

Variables

- const uint8_t **I2C_ADDR** = 0x27

  Endereço padrão do módulo I2C do LCD.
- const uint8_t **LCD_COLUMNS** = 16

  Quantidade de colunas do LCD.
- const uint8_t **LCD_LINES** = 2

  Quantidade de linhas do LCD.
- float **lastTemperature** = -1000.0

  Último valor de temperatura exibido.
- float **lastHumidity** = -1000.0

  Último valor de umidade exibido.
- const float TEMP_UPDATE_THRESHOLD = 0.01

  Limiar mínimo para atualização da temperatura.
- const float **HUMI_UPDATE_THRESHOLD** = 0.01

  Limiar mínimo para atualização da umidade.
- bool **isBacklightTurnOn** = true

  Indica se o backlight está ligado.
- unsigned long **lastTimeBackligthTurnOn** = 0

  Guarda o tempo da última ativação do backlight.

4.12.1 Detailed Description

Implementação do controle do display LCD 1602 via I2C.

Inclui a biblioteca padrão para comunicação I2C, a biblioteca de controle do display LCD 1602, e funções matemáticas utilizadas para cálculo de valor absoluto.

Este módulo implementa funções para:

- Inicializar o display
- Limpar linhas
- Exibir temperatura e umidade
- Controlar o backlight

4.12.2 Function Documentation

4.12.2.1 clearLine()

```
void clearLine (
            uint8_t line)
```

Limpa uma linha do display.

Parameters

line	Linha do display a ser limpa (uint8_t).

4.12.2.2 handleBacklightLCD()

```
void handleBacklightLCD (
            bool isAlertState,
            bool commandTurnOnLCD)
```

Controla o estado do backlight.

Controla o estado da luz de fundo do display LCD.

Parameters

isAlertState	Indica alerta.
commandTurnOnLCD	Comando manual.

4.12.2.3 lcd1602_init()

```
void lcd1602_init ()
```

Inicializa o display.

Inicializa o display LCD 1602 via I2C.

Inicializa comunicação I2C, ativa o backlight e exibe mensagem inicial.

4.12.2.4 lcd1602_showData()

```
void lcd1602_showData (
            float temp,
            float humi,
            bool alertTemp,
            bool alertHumi)
```

Exibe os valores de temperatura e umidade no display LCD.

Esta função atualiza o display com os valores atuais de temperatura e umidade, podendo indicar visualmente situações de alerta quando os limites configurados são ultrapassados.

Parameters

temp	Valor da temperatura em graus Celsius.
humi	Valor da umidade relativa do ar em porcentagem.
alertTemp	Indica condição de alerta para temperatura.
alertHumi	Indica condição de alerta para umidade.

4.12.2.5 show_value()

```
void show_value (
            uint8_t line,
            const __FlashStringHelper * normalMsg,
            const __FlashStringHelper * alertMsg,
            float value,
            bool alert,
            const char * unit)
```

Exibe no display o valor de temperatura ou umidade.

Função genérica responsável por mostrar uma mensagem e o valor correspondente em uma linha específica do display, podendo alternar entre mensagem normal e mensagem de alerta.

Parameters

line	Linha do display (0 = primeira linha, 1 = segunda linha).
normalMsg	Ponteiro para mensagem normal armazenada na memória Flash.
alertMsg	Ponteiro para mensagem de alerta armazenada na memória Flash.
value	Valor de temperatura ou umidade a ser exibido.
alert	Define se deve exibir a mensagem de alerta (true) ou normal (false).
unit	Sufixo da unidade de medida (ex.: "%" ou "\xDF""C").

4.12.3 Variable Documentation

4.12.3.1 TEMP_UPDATE_THRESHOLD

```
const float TEMP_UPDATE_THRESHOLD = 0.01
```

Limiar mínimo para atualização da temperatura.

Evita reescrever o display sem necessidade.

4.13 display_LCD-1602_I2C.h

```
#ifndef DISPLAY_LCD_1602_I2C_H
#define DISPLAY_LCD_1602_I2C_H


extern void lcd1602_init();


extern void lcd1602_showData(float temp, float humi, bool alertTemp, bool alertHumi);


extern void handleBacklightLCD(bool isAlertState, bool commandTurnOnLCD);


#endif
```

4.14 microcontrolador/led.cpp File Reference

Implementação das funções de controle do LED indicador do sistema.

```
#include "led.h"
#include "config.h"
```

Functions

- void ledInit ()

  Inicializa o hardware do LED.
- void **toggleLed** (unsigned long now)
- void turnOnLed ()

  Liga o LED.
- void turnOffLed ()

  Desliga o LED.

Variables

- unsigned long lastTimeLedTurnedOn = 0

  Armazena o instante da última alternância do LED.
- constexpr unsigned long LED_INTERVAL_MS = 2000

  Intervalo de tempo para alternância do LED.

4.14.1 Detailed Description

Implementação das funções de controle do LED indicador do sistema.

Este arquivo contém a implementação das funções responsáveis por inicializar, ligar, desligar e alternar o estado do LED.

O controle é feito de forma não bloqueante utilizando a função millis(), permitindo que o LED pisque sem interromper a execução das demais tarefas do sistema.

O LED pode ser usado para:

- Indicação de funcionamento
- Sinalização de alerta
- Debug visual do sistema

4.14.2 Function Documentation

4.14.2.1 ledInit()

```
void ledInit ()
```

Inicializa o hardware do LED.

Configura o pino definido em PIN_LED como saída digital e garante que o LED inicie desligado.

Esta função deve ser chamada uma única vez durante a inicialização do sistema, normalmente dentro da função setup().

Note

Evita estados indefinidos no boot.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/led.cpp, and C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/led.h.

4.14.2.2 turnOffLed()

```
void turnOffLed ()
```

Desliga o LED.

Força o pino do LED para nível lógico LOW.

Note

Pode ser usado para economizar energia.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/led.cpp, and C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/led.h.

4.14.2.3 turnOnLed()

```
void turnOnLed ()
```

Liga o LED.

Força o pino do LED para nível lógico HIGH.

Note

Usado para sinalização contínua.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/led.cpp, and C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/led.h.

4.14.3 Variable Documentation

4.14.3.1 lastTimeLedTurnedOn

```
unsigned long lastTimeLedTurnedOn = 0
```

Armazena o instante da última alternância do LED.

Esta variável é usada como referência para calcular o tempo decorrido desde a última mudança de estado do LED.

Note

Utilizada para controle não bloqueante baseado em millis().

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/↩ microcontrolador/led.cpp.

4.14.3.2 LED_INTERVAL_MS

```
unsigned long LED_INTERVAL_MS = 2000  [constexpr]
```

Intervalo de tempo para alternância do LED.

Define o período de piscar do LED em milissegundos.

Valor atual:

- 2000 ms = 2 segundos

Note

constexpr garante valor constante em tempo de compilação.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/↩ microcontrolador/led.cpp.

4.15 microcontrolador/led.h File Reference

Interface do módulo de controle do LED indicador.

```
#include <Arduino.h>
```

Functions

- void ledInit ()

  Inicializa o hardware do LED.
- void **toggleLed** (unsigned long now)
- void turnOnLed ()

  Liga o LED.
- void turnOffLed ()

  Desliga o LED.

4.15.1 Detailed Description

Interface do módulo de controle do LED indicador.

Este arquivo define a interface pública para controle do LED do sistema.

O LED pode ser utilizado para:

- Indicação de funcionamento normal
- Sinalização de estados de alerta
- Feedback visual para o usuário
- Auxílio em debug

O controle é realizado de forma não bloqueante utilizando temporização baseada na função millis().

Note

A implementação está disponível em led.cpp

4.15.2 Function Documentation

4.15.2.1 ledInit()

```
void ledInit ()
```

Inicializa o hardware do LED.

Configura o pino do LED como saída digital e garante que ele inicie desligado.

Esta função deve ser chamada uma única vez durante a inicialização do sistema, normalmente dentro da função setup().

```
void setup()
{
    ledInit();
}
```

Note

Evita estados indefinidos no momento da energização.

Configura o pino definido em PIN_LED como saída digital e garante que o LED inicie desligado.

Esta função deve ser chamada uma única vez durante a inicialização do sistema, normalmente dentro da função setup().

Note

Evita estados indefinidos no boot.

4.15.2.2 turnOffLed()

```
void turnOffLed ()
```

Desliga o LED.

Força o pino do LED para nível lógico LOW.

Note

Pode ser usado para economizar energia.

4.15.2.3 turnOnLed()

```
void turnOnLed ()
```

Liga o LED.

Força o pino do LED para nível lógico HIGH.

Note

Usado para sinalização contínua.

4.16 led.h

Go to the documentation of this file.

```
00001
00019
00020 #ifndef LED_H
00021 #define LED_H
00022 #include <Arduino.h>
00023
00024
00025
00026 // ---------------------------------------------------------------------------
00027 // Funções públicas do módulo
00028 // ---------------------------------------------------------------------------
00029
00047 void ledInit();
00048
00049
00075 void toggleLed(unsigned long now);
00076
00077
00090 void turnOnLed();
00091
00092
00109 void turnOffLed();
00110
00111
00112 #endif
```

4.17 microcontrolador/log.cpp File Reference

Implementação de funções de logging para depuração e monitoramento do sistema.

```
#include "log.h"
#include <Arduino.h>
#include <stdarg.h>
```

Functions

- void logInit (LogLevel level)

  Inicializa o sistema de logs.
- void log (LogLevel level, const char *fmt,...)

  Registra uma mensagem de log com formato customizado.

Variables

- LogLevel CURRENT_LOG_LEVEL = LOG_DEBUG

  Variável global que armazena o nível de log atual.

4.17.1 Detailed Description

Implementação de funções de logging para depuração e monitoramento do sistema.

Este módulo fornece um mecanismo centralizado para registrar mensagens no Serial em diferentes níveis de severidade (ERRO, WARN, INFO, DEBUG).

4.17.2 Function Documentation

4.17.2.1 log()

```
void log (
            LogLevel level,
            const char * fmt,
             ...)
```

Registra uma mensagem de log com formato customizado.

Registra uma mensagem no log.

Esta função formata a mensagem recebida e imprime na Serial, respeitando o nível atual de log definido.

Parameters

level	Nível da mensagem (ERRO, WARN, INFO, DEBUG).
fmt	String de formato (similar ao printf).
...	Argumentos variáveis correspondentes ao formato.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/↩ microcontrolador/mqtt.cpp.

4.17.2.2 logInit()

```
void logInit (
            LogLevel level)
```

Inicializa o sistema de logs.

Inicializa o sistema de log.

- Define o nível de log inicial.
- Inicia a comunicação Serial a 115200 baud.
- Registra uma mensagem indicando que o log foi iniciado.

Parameters

level	Nível de log a ser utilizado (LOG_ERROR, LOG_WARN, LOG_INFO, LOG_DEBUG).

4.17.3 Variable Documentation

4.17.3.1 CURRENT_LOG_LEVEL

```
LogLevel CURRENT_LOG_LEVEL = LOG_DEBUG
```

Variável global que armazena o nível de log atual.

Controla quais mensagens serão exibidas. Deve ser inicializada pela função logInit().

4.18 microcontrolador/log.h File Reference

Declarações para o sistema de logging do projeto.

```
#include <Arduino.h>
#include <stdarg.h>
```

Enumerations

- enum LogLevel {
  LOG_NONE = 0 , LOG_ERROR = 1 , LOG_WARN = 2 , LOG_INFO = 3 ,
  LOG_DEBUG = 4 }

  Define os níveis de log disponíveis.

Functions

- void logInit (LogLevel level)

  Inicializa o sistema de log.
- void log (LogLevel level, const char *fmt,...)

  Registra uma mensagem no log.

Variables

- LogLevel CURRENT_LOG_LEVEL

  Variável global que armazena o nível de log atual.

4.18.1 Detailed Description

Declarações para o sistema de logging do projeto.

Este módulo define níveis de log e funções utilitárias para registrar mensagens em diferentes níveis de severidade. Pode ser configurado para controlar a quantidade de informações enviadas à saída serial durante a execução.

4.18.2 Enumeration Type Documentation

4.18.2.1 LogLevel

```
enum LogLevel
```

Define os níveis de log disponíveis.

Enumerator

LOG_NONE	Nenhum log será exibido
LOG_ERROR	Mensagens de erro (alta prioridade)
LOG_WARN	Mensagens de aviso
LOG_INFO	Mensagens informativas
LOG_DEBUG	Mensagens de depuração (mais verboso)

4.18.3 Function Documentation

4.18.3.1 log()

```
void log (
            LogLevel level,
            const char * fmt,
             ...)
```

Registra uma mensagem no log.

Exibe a mensagem apenas se o nível informado for maior ou igual ao nível global configurado em CURRENT_↩ LOG_LEVEL.

Parameters

level	Nível da mensagem (LOG_ERROR, LOG_WARN, LOG_INFO, LOG_DEBUG).
fmt	String formatada no estilo printf.

...	Argumentos variáveis para formatação.

Registra uma mensagem no log.

Esta função formata a mensagem recebida e imprime na Serial, respeitando o nível atual de log definido.

Parameters

level	Nível da mensagem (ERRO, WARN, INFO, DEBUG).
fmt	String de formato (similar ao printf).
...	Argumentos variáveis correspondentes ao formato.

4.18.3.2 logInit()

```
void logInit (
            LogLevel level)
```

Inicializa o sistema de log.

Configura o nível de log inicial e inicia a comunicação serial.

Parameters

level	Nível de log desejado (LOG_ERROR, LOG_WARN, LOG_INFO, LOG_DEBUG).

Inicializa o sistema de log.

- Define o nível de log inicial.
- Inicia a comunicação Serial a 115200 baud.
- Registra uma mensagem indicando que o log foi iniciado.

Parameters

level	Nível de log a ser utilizado (LOG_ERROR, LOG_WARN, LOG_INFO, LOG_DEBUG).

4.18.4 Variable Documentation

4.18.4.1 CURRENT_LOG_LEVEL

```
LogLevel CURRENT_LOG_LEVEL  [extern]
```

Variável global que armazena o nível de log atual.

Controla quais mensagens serão exibidas. Deve ser inicializada pela função logInit().

4.19 log.h

Go to the documentation of this file.

```
#ifndef LOG_H
#define LOG_H

#include <Arduino.h>
#include <stdarg.h>


enum LogLevel {
  LOG_NONE = 0,
  LOG_ERROR = 1,
  LOG_WARN = 2,
  LOG_INFO = 3,
  LOG_DEBUG = 4
};

extern LogLevel CURRENT_LOG_LEVEL;

void logInit(LogLevel level);

void log(LogLevel level, const char* fmt, ...);

#endif
```

4.20 microcontrolador/mqtt.cpp File Reference

Implementação do módulo de comunicação MQTT.

```
#include "config.h"
#include "mqtt.h"
#include "log.h"
#include <WiFiClientSecure.h>
#include <ArduinoJson.h>
#include "storage.h"
#include "config_storage.h"
```

Functions

- PubSubClient client (espClient)

  Cliente MQTT responsável pela comunicação com o broker.
- void **setupMQTT** ()
- void resetMQTTClient ()

  Reinicializa o cliente MQTT e o cliente de rede.
- bool **checkMQTTConnected** ()
- bool publishSensorData (float temperature, float humidity)

  Publica dados de sensores no broker MQTT.
- void resendMqttData ()

  Reenvia dados armazenados localmente.
- void publishAlert (float temperature, float humidity)

  Publica mensagem de alerta no broker MQTT.
- void resetStateSendAlert ()

  Reseta o estado de envio de alerta.

Variables

- WiFiClient espClient

  Cliente de rede utilizado pelo MQTT.
- bool wasSendAlert = false

  Indica se um alerta já foi enviado.

4.20.1 Detailed Description

Implementação do módulo de comunicação MQTT.

Este módulo é responsável por gerenciar toda a comunicação entre o dispositivo e o broker MQTT, incluindo:

- Configuração da conexão com o broker;
- Gerenciamento de reconexão automática;
- Envio de dados de sensores em formato JSON;
- Reenvio de dados armazenados localmente em caso de falha;
- Publicação de alertas;
- Recuperação do cliente em caso de falhas críticas.

A comunicação é baseada na biblioteca PubSubClient.

O módulo foi projetado para operar de forma não bloqueante utilizando millis().

4.20.2 Function Documentation

4.20.2.1 client()

```
PubSubClient client (
            espClient )
```

Cliente MQTT responsável pela comunicação com o broker.

Utiliza o cliente de rede espClient como camada de transporte.

4.20.2.2 publishAlert()

```
void publishAlert (
            float temperature,
            float humidity)
```

Publica mensagem de alerta no broker MQTT.

Utilizado quando o sistema detecta condições críticas.

O envio ocorre apenas uma vez até ser resetado.

Parameters

temperature	Temperatura atual
humidity	Umidade atual

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.cpp, and C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.h.

4.20.2.3 publishSensorData()

```
bool publishSensorData (
            float temperature,
            float humidity)
```

Publica dados de sensores no broker MQTT.

Os dados são enviados no formato JSON.

Estrutura:

```
{
  "microcontrollerId": "...",
  "temperature": 25.0,
  "humidity": 60.0
}
```

Parameters

temperature	Temperatura em graus Celsius
humidity	Umidade relativa do ar

Returns

true Sucesso

false Falha

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.cpp, and C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.h.

4.20.2.4 resendMqttData()

```
void resendMqttData ()
```

Reenvia dados armazenados localmente.

Reenvia mensagens armazenadas localmente.

Utilizado quando houve falha de comunicação anterior.

Fluxo:

- Verifica armazenamento;
- Envia mensagem;
- Remove mensagem enviada.

Note

Utiliza LittleFS como armazenamento.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.cpp, and C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.h.

4.20.2.5 resetMQTTClient()

```
void resetMQTTClient ()
```

Reinicializa o cliente MQTT e o cliente de rede.

Utilizado quando ocorrem múltiplas falhas de conexão.

Esta função recria:

- Cliente de rede
- Cliente MQTT

Permitindo restaurar a comunicação sem reiniciar o dispositivo.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.cpp.

4.20.2.6 resetStateSendAlert()

```
void resetStateSendAlert ()
```

Reseta o estado de envio de alerta.

Permite que novos alertas sejam enviados.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.cpp, and C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.h.

4.20.3 Variable Documentation

4.20.3.1 espClient

```
WiFiClient espClient
```

Cliente de rede utilizado pelo MQTT.

Responsável pela comunicação TCP/IP com o broker.

Note

Pode ser substituído por WiFiClientSecure caso TLS seja utilizado.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.cpp.

4.20.3.2 wasSendAlert

```
bool wasSendAlert = false
```

Indica se um alerta já foi enviado.

Evita envio repetido da mesma mensagem de alerta.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.cpp.

4.21 microcontrolador/mqtt.h File Reference

Interface pública do módulo de comunicação MQTT.

```
#include <PubSubClient.h>
```

Functions

- void **setupMQTT** ()
- bool **checkMQTTConnected** ()
- bool publishSensorData (float temperature, float humidity)

  Publica dados de sensores no broker MQTT.
- void resendMqttData ()

  Reenvia mensagens armazenadas localmente.
- void publishAlert (float temperature, float humidity)

  Publica mensagem de alerta no broker MQTT.
- void resetStateSendAlert ()

  Reseta o estado de envio de alerta.

Variables

- PubSubClient client

  Cliente MQTT global.

4.21.1 Detailed Description

Interface pública do módulo de comunicação MQTT.

Este módulo fornece os recursos necessários para comunicação entre o microcontrolador e um broker MQTT.

Funcionalidades principais:

- Inicialização da conexão MQTT;
- Gerenciamento automático de reconexão;
- Publicação de dados de sensores;
- Publicação de mensagens de alerta;
- Reenvio automático de mensagens armazenadas localmente.

O envio de dados é realizado utilizando payloads no formato JSON.

Note

A implementação está localizada em mqtt.cpp

4.21.2 Function Documentation

4.21.2.1 publishAlert()

```
void publishAlert (
            float temperature,
            float humidity)
```

Publica mensagem de alerta no broker MQTT.

Utilizado quando o sistema detecta condições críticas.

Exemplo de uso:
`publishAlert(temp, hum);`

Parameters

temperature	Temperatura atual
humidity	Umidade atual

Note

O alerta é enviado apenas uma vez até ser resetado.

Utilizado quando o sistema detecta condições críticas.

O envio ocorre apenas uma vez até ser resetado.

Parameters

temperature	Temperatura atual
humidity	Umidade atual

4.21.2.2 publishSensorData()

```
bool publishSensorData (
            float temperature,
            float humidity)
```

Publica dados de sensores no broker MQTT.

Os dados são enviados no formato JSON contendo:

- Identificador do dispositivo
- Temperatura
- Umidade

Exemplo de payload:

```
{
  "microcontrollerId": "device01",
  "temperature": 25.4,
  "humidity": 60.1
}
```

Parameters

temperature	Temperatura em graus Celsius.
humidity	Umidade relativa em porcentagem.

Returns

true Sucesso no envio

false Falha no envio

Note

Em caso de falha, os dados podem ser armazenados localmente.

Os dados são enviados no formato JSON.

Estrutura:

```
{
  "microcontrollerId": "...",
  "temperature": 25.0,
  "humidity": 60.0
}
```

Parameters

temperature	Temperatura em graus Celsius
humidity	Umidade relativa do ar

Returns

true Sucesso

false Falha

4.21.2.3 resendMqttData()

```
void resendMqttData ()
```

Reenvia mensagens armazenadas localmente.

Esta função:

- Verifica se existem mensagens armazenadas;
- Envia uma mensagem por vez;
- Remove mensagens enviadas com sucesso.

Utilizado para garantir integridade dos dados quando há falha de conexão.

Note

Deve ser chamada periodicamente dentro do loop principal.

Reenvia mensagens armazenadas localmente.

Utilizado quando houve falha de comunicação anterior.

Fluxo:

- Verifica armazenamento;
- Envia mensagem;
- Remove mensagem enviada.

Note

Utiliza LittleFS como armazenamento.

4.21.2.4 resetStateSendAlert()

```
void resetStateSendAlert ()
```

Reseta o estado de envio de alerta.

Permite que novos alertas sejam enviados.

4.21.3 Variable Documentation

4.21.3.1 client

```
PubSubClient client  [extern]
```

Cliente MQTT global.

Responsável pela comunicação com o broker MQTT.

Este objeto é:

- Inicializado em mqtt.cpp
- Utilizado internamente pelas funções deste módulo

Note

Pode ser utilizado diretamente em casos avançados.

Warning

Evite modificar suas configurações fora do módulo MQTT.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/↩ microcontrolador/mqtt.cpp, and C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/↩ Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.h.

4.22 mqtt.h

Go to the documentation of this file.

```
#ifndef MQTT_HELPER_H
#define MQTT_HELPER_H

#include <PubSubClient.h>



// ---------------------------------------------------------------------------
// Variáveis globais
// ---------------------------------------------------------------------------

extern PubSubClient client;


// ---------------------------------------------------------------------------
// Funções públicas
// ---------------------------------------------------------------------------

void setupMQTT();


bool checkMQTTConnected();


bool publishSensorData(float temperature, float humidity);


void resendMqttData();


void publishAlert(float temperature, float humidity);


void resetStateSendAlert();


#endif
```

4.23 microcontrolador/sensor.cpp File Reference

Implementação do módulo de leitura do sensor ambiental AHT10.

```
#include <Wire.h>
#include "sensor.h"
#include "log.h"
#include "config.h"
```

Functions

- void initializeSensor ()

  Inicializa o sensor AHT10.
- EnvironmentData readSensorData ()

  Lê os dados de temperatura e umidade do sensor AHT10.

Variables

- Adafruit_AHTX0 aht

  Objeto global do sensor AHT10 (via biblioteca Adafruit).

4.23.1 Detailed Description

Implementação do módulo de leitura do sensor ambiental AHT10.

Este módulo é responsável por inicializar e realizar a leitura dos dados de temperatura e umidade relativa do ar utilizando o sensor AHT10 através da interface de comunicação I^2C.

Funcionalidades principais:

- Inicialização do barramento I^2C;
- Inicialização do sensor AHT10;
- Leitura periódica dos dados ambientais;
- Validação dos dados lidos;
- Fornecimento dos dados através da estrutura EnvironmentData.

Os dados coletados são utilizados por outros módulos do sistema, como:

- Exibição no display LCD;
- Envio para o broker MQTT;
- Verificação de condições de alerta;
- Armazenamento local.

Comunicação:

- Interface: I^2C
- Pinos utilizados: SDA → definido em config.h SCL → definido em config.h

Biblioteca utilizada:

- Adafruit AHTX0

Estrutura de dados retornada:

```
struct EnvironmentData {
    float temperature;
    float humidity;
    bool valid;
};
```

Note

Este módulo depende da inicialização correta do barramento I^2C.

Warning

Caso o sensor não seja inicializado corretamente, os dados retornados serão inválidos.

See also

sensor.h

4.23.2 Function Documentation

4.23.2.1 initializeSensor()

```
void initializeSensor ()
```

Inicializa o sensor AHT10.

- Configura a comunicação I^2C nos pinos definidos (SDA = 4, SCL = 5).
- Tenta iniciar o sensor AHT10 utilizando a biblioteca Adafruit.
- Caso não consiga inicializar, registra mensagem de erro no log.
- Caso seja bem-sucedido, registra mensagem de debug.

Note

Em versões futuras, os pinos podem ser passados como parâmetros em vez de fixos.

4.23.2.2 readSensorData()

```
EnvironmentData readSensorData ()
```

Lê os dados de temperatura e umidade do sensor AHT10.

- Captura os eventos de temperatura e umidade através da biblioteca Adafruit.
- Armazena os valores na estrutura `EnvironmentData`.
- Realiza validações para garantir que os dados são numéricos e plausíveis.

Returns

Estrutura `EnvironmentData` contendo:

- `temperature` → Temperatura em graus Celsius.
- `humidity` → Umidade relativa em %.
- `valid` → `true` se os dados são válidos, `false` caso contrário.

4.23.3 Variable Documentation

4.23.3.1 aht

```
Adafruit_AHTX0 aht
```

Objeto global do sensor AHT10 (via biblioteca Adafruit).

Objeto global do sensor AHT10.

4.24 microcontrolador/sensor.h File Reference

Declarações relacionadas ao uso do sensor AHT10 (temperatura e umidade).

```
#include <Adafruit_AHTX0.h>
```

Classes

- struct EnvironmentData

  Estrutura que armazena os dados ambientais coletados do sensor.

Functions

- void initializeSensor ()

  Inicializa o sensor AHT10.
- EnvironmentData readSensorData ()

  Lê os dados de temperatura e umidade do sensor AHT10.

Variables

- Adafruit_AHTX0 aht

  Objeto global do sensor AHT10.

4.24.1 Detailed Description

Declarações relacionadas ao uso do sensor AHT10 (temperatura e umidade).

Este módulo encapsula a inicialização e a leitura de dados do sensor AHT10, retornando os valores em uma estrutura própria.

4.24.2 Function Documentation

4.24.2.1 initializeSensor()

```
void initializeSensor ()
```

Inicializa o sensor AHT10.

- Configura a comunicação I^2C.
- Tenta iniciar o sensor AHT10.
- Caso a inicialização falhe, gera uma mensagem de erro via log.
- Configura a comunicação I^2C nos pinos definidos (SDA = 4, SCL = 5).
- Tenta iniciar o sensor AHT10 utilizando a biblioteca Adafruit.
- Caso não consiga inicializar, registra mensagem de erro no log.
- Caso seja bem-sucedido, registra mensagem de debug.

Note

Em versões futuras, os pinos podem ser passados como parâmetros em vez de fixos.

4.24.2.2 readSensorData()

```
EnvironmentData readSensorData ()
```

Lê os dados de temperatura e umidade do sensor AHT10.

- Retorna uma estrutura `EnvironmentData` contendo os valores lidos.
- Caso os dados não sejam válidos (NaN ou valores muito baixos), o campo `valid` será `false`.

Returns

Estrutura `EnvironmentData` com temperatura, umidade e validade dos dados.

- Captura os eventos de temperatura e umidade através da biblioteca Adafruit.
- Armazena os valores na estrutura `EnvironmentData`.
- Realiza validações para garantir que os dados são numéricos e plausíveis.

Returns

Estrutura `EnvironmentData` contendo:

- `temperature` → Temperatura em graus Celsius.
- `humidity` → Umidade relativa em %.
- `valid` → `true` se os dados são válidos, `false` caso contrário.

4.24.3 Variable Documentation

4.24.3.1 aht

```
Adafruit_AHTX0 aht  [extern]
```

Objeto global do sensor AHT10.

Declarado como `extern` para ser definido em `sensor.cpp` e acessível em outros módulos.

Objeto global do sensor AHT10.

4.25 sensor.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef SENSOR_H
00010 #define SENSOR_H
00011
00012 #include <Adafruit_AHTX0.h>
00013
00014
00024 struct EnvironmentData {
00025   float temperature;
00026   float humidity;
00027   bool valid;
00028 };
00029
00037 void initializeSensor();
00038
00047 EnvironmentData readSensorData();
00048
00054 extern Adafruit_AHTX0 aht;
00055
00056 #endif
```

4.26 microcontrolador/server.h File Reference

Interface do módulo responsável pelo servidor HTTP embarcado no ESP8266.

```
#include <ESP8266WebServer.h>
```

Functions

- void startServer (ESP8266WebServer *server)

  Inicia o servidor HTTP e configura as rotas de acesso.
- void stopServer (ESP8266WebServer *server)

  Encerra o servidor HTTP e desconecta o Access Point.
- void server_setup_routes (ESP8266WebServer *server)

  Configura as rotas HTTP do servidor.
- void server_handle_loop (ESP8266WebServer *server)

  Mantém o servidor ativo e processa requisições de clientes.

4.26.1 Detailed Description

Interface do módulo responsável pelo servidor HTTP embarcado no ESP8266.

Este módulo fornece funções para criação, configuração, execução e encerramento de um servidor web HTTP no microcontrolador ESP8266.

O servidor é utilizado principalmente para:

- Disponibilizar uma interface web de configuração do dispositivo;
- Receber parâmetros enviados pelo usuário através de formulários HTML;
- Permitir configuração de rede Wi-Fi, parâmetros MQTT e limites ambientais;
- Controlar o ciclo de vida do servidor durante o modo de configuração.

Funcionalidades principais:

- Inicialização do servidor HTTP;
- Registro de rotas e endpoints;
- Processamento de requisições de clientes;
- Encerramento seguro do servidor;
- Suporte ao modo Access Point (AP) para configuração local.

Fluxo típico de utilização:

```
ESP8266WebServer server(80);

startServer(&server);

void loop() {
    server_handle_loop(&server);
}
```

Dependências:

- Biblioteca ESP8266WebServer
- Módulo de configuração do sistema

Note

Este módulo é utilizado principalmente durante o modo de configuração do dispositivo.

Warning

O servidor deve ser processado continuamente no loop principal para garantir o atendimento das requisições HTTP.

See also

server.cpp

4.26.2 Function Documentation

4.26.2.1 server_handle_loop()

```
void server_handle_loop (
            ESP8266WebServer * server)
```

Mantém o servidor ativo e processa requisições de clientes.

Essa função deve ser chamada no loop principal do programa (`loop()`), garantindo que o servidor trate novas conexões e requisições HTTP.

Parameters

server	Ponteiro para a instância de ESP8266WebServer em execução.

Mantém o servidor ativo e processa requisições de clientes.

Deve ser chamada dentro do `loop()` principal para processar requisições de clientes conectados.

Parameters

server	Ponteiro para a instância de `ESP8266WebServer`.

4.26.2.2 server_setup_routes()

```
void server_setup_routes (
            ESP8266WebServer * server)
```

Configura as rotas HTTP do servidor.

Define as páginas e endpoints que o servidor responderá. Normalmente, inclui a rota raiz ("/") e a rota de envio de formulário ("/submit").

Parameters

server	Ponteiro para a instância de ESP8266WebServer onde as rotas serão registradas.

4.26.2.3 startServer()

```
void startServer (
            ESP8266WebServer * server)
```

Inicia o servidor HTTP e configura as rotas de acesso.

Essa função chama as rotas definidas em server_setup_routes() e inicia o serviço do servidor. Deve ser chamada após configurar o modo Access Point (AP) ou conectar o dispositivo a uma rede Wi-Fi.

Parameters

server	Ponteiro para a instância de ESP8266WebServer a ser iniciada.

Inicia o servidor HTTP e configura as rotas de acesso.

Esta função garante que o servidor só será iniciado uma vez. Chama `serverSetupRoutes()` para registrar os endpoints e inicia o servidor com `server->begin()`.

Parameters

server	Ponteiro para a instância de `ESP8266WebServer`.

4.26.2.4 stopServer()

```
void stopServer (
            ESP8266WebServer * server)
```

Encerra o servidor HTTP e desconecta o Access Point.

Essa função interrompe o atendimento de requisições HTTP, encerra a conexão com os clientes e desativa o modo AP do Wi-Fi.

Parameters

server	Ponteiro para a instância de ESP8266WebServer a ser parada.

Encerra o servidor HTTP e desconecta o Access Point.

Esta função interrompe conexões ativas, encerra o serviço HTTP e retorna o módulo Wi-Fi para o modo estação (**WIFI_STA**).

Parameters

server	Ponteiro para a instância de `ESP8266WebServer`.

4.27 server.h

Go to the documentation of this file.

```
00001
00047
00048 #ifndef SERVER_H
00049 #define SERVER_H
00050
00051 #include <ESP8266WebServer.h>  // Biblioteca para criar e gerenciar um servidor HTTP no ESP8266
00052
00062 void startServer(ESP8266WebServer *server);
00063
00072 void stopServer(ESP8266WebServer *server);
00073
00082 void server_setup_routes(ESP8266WebServer* server);
00083
00092 void server_handle_loop(ESP8266WebServer* server);
00093
00094 #endif
```

4.28 microcontrolador/storage.cpp File Reference

Implementação do módulo de armazenamento persistente de dados de sensores utilizando LittleFS.

```
#include "storage.h"
#include "log.h"
#include <ArduinoJson.h>
#include <LittleFS.h>
```

Functions

- void initStorage ()

  Inicializa o sistema de arquivos LittleFS.
- void saveStorage (float temperature, float humidity)

  Salva um novo objeto (temperatura e umidade) no arquivo JSON.
- ObjectStorage getObjectStorage (size_t index)

  Recupera um objeto armazenado em uma linha específica do arquivo JSON.
- void deleteOneMessage ()

  Remove a primeira mensagem armazenada no arquivo.
- bool hasDataStorage ()

  Verifica se há objetos armazenados.
- void storagePrint ()

  Imprime no Serial todos os objetos armazenados no arquivo.

Variables

- const char ∗ **PATH** = "/storage.json"

  Caminho do arquivo de armazenamento no sistema de arquivos LittleFS.
- bool **hasObj** = false

  Flag que indica se há algum objeto armazenado no sistema.

4.28.1 Detailed Description

Implementação do módulo de armazenamento persistente de dados de sensores utilizando LittleFS.

Este módulo é responsável por gerenciar o armazenamento local dos dados ambientais (temperatura e umidade) no sistema de arquivos LittleFS do microcontrolador.

O armazenamento é utilizado como mecanismo de tolerância a falhas de comunicação, permitindo salvar temporariamente os dados quando não for possível enviá-los ao servidor remoto (por exemplo, falha na conexão MQTT).

Funcionalidades principais:

- Inicialização do sistema de arquivos LittleFS;
- Armazenamento persistente de dados de sensores em formato JSON;
- Recuperação de dados armazenados por índice;
- Remoção sequencial de dados já transmitidos;
- Verificação da existência de dados pendentes;
- Impressão dos dados armazenados para fins de depuração.

Estrutura de armazenamento:

- Os dados são armazenados no arquivo `/storage.json`;
- Cada linha do arquivo representa um objeto JSON independente;
- Cada objeto contém:

```
{
  "t": 25.40,
  "h": 60.20
}
```

Estratégia de funcionamento:

1. Quando ocorre falha no envio MQTT: → Os dados são salvos localmente utilizando saveStorage();
2. Quando a conexão é restabelecida: → Os dados são recuperados com getObjectStorage(); → Enviados ao servidor; → Removidos com deleteOneMessage();

Benefícios desta abordagem:

- Evita perda de dados;
- Garante entrega confiável;
- Implementa buffer persistente;
- Suporte a reconexão automática.

Dependências:

- LittleFS (Sistema de arquivos)
- ArduinoJson (Serialização JSON)
- log.h (Sistema de logging)

Arquivo relacionado:

See also

storage.h

Note

O armazenamento é baseado em append, garantindo maior eficiência e menor desgaste da memória Flash.

Warning

O LittleFS deve ser inicializado com initStorage() antes de qualquer operação.

4.28.2 Function Documentation

4.28.2.1 deleteOneMessage()

```
void deleteOneMessage ()
```

Remove a primeira mensagem armazenada no arquivo.

Remove o primeiro registro do storage.

- Cria um arquivo temporário.
- Copia todas as linhas do arquivo original, exceto a primeira.
- Substitui o arquivo original pelo temporário.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/↩ microcontrolador/mqtt.cpp.

4.28.2.2 getObjectStorage()

```
ObjectStorage getObjectStorage (
            size_t index)
```

Recupera um objeto armazenado em uma linha específica do arquivo JSON.

Obtém um registro do storage.

- Cada linha do arquivo representa um objeto JSON.
- A leitura é feita até alcançar o índice informado.
- Caso os dados sejam válidos, retorna um objeto preenchido.
- Caso contrário, retorna um objeto inválido (`valid = false`).

Parameters

index	Índice (linha) do objeto a ser recuperado.

Returns

Estrutura `ObjectStorage` contendo temperatura, umidade e validade.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/↩ microcontrolador/mqtt.cpp.

4.28.2.3 hasDataStorage()

```
bool hasDataStorage ()
```

Verifica se há objetos armazenados.

Verifica se existem registros armazenados.

Returns

true se existe pelo menos um objeto armazenado.

false caso contrário.

Examples

C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/↩ microcontrolador/mqtt.cpp.

4.28.2.4 initStorage()

```
void initStorage ()
```

Inicializa o sistema de arquivos LittleFS.

Inicializa o sistema de armazenamento.

- Monta a partição LittleFS.
- Caso a montagem falhe, gera um log de erro.

4.28.2.5 saveStorage()

```
void saveStorage (
            float temperature,
            float humidity)
```

Salva um novo objeto (temperatura e umidade) no arquivo JSON.

Salva um novo registro de temperatura e umidade no storage.

- Abre o arquivo em modo append (`"a"`).
- Cria um documento JSON com os valores fornecidos.
- Grava os dados no arquivo e fecha.
- Atualiza a flag `hasObj`.

Parameters

temperature	Valor da temperatura a ser armazenada.
humidity	Valor da umidade a ser armazenada.

4.28.2.6 storagePrint()

```
void storagePrint ()
```

Imprime no Serial todos os objetos armazenados no arquivo.

Imprime todo o conteúdo do storage no Serial.

- Lê o arquivo linha por linha.
- Desserializa cada linha em JSON.
- Exibe temperatura e umidade formatadas no console.

4.29 microcontrolador/storage.h File Reference

Interface pública do módulo de armazenamento persistente utilizando LittleFS.

```
#include <Arduino.h>
```

Classes

- struct ObjectStorage

  Estrutura que representa um registro armazenado no sistema.

Functions

- void initStorage ()

  Inicializa o sistema de armazenamento.
- void saveStorage (float temperature, float humidity)

  Salva um novo registro de temperatura e umidade no storage.
- ObjectStorage getObjectStorage (size_t index=0)

  Obtém um registro do storage.
- void storagePrint ()

  Imprime todo o conteúdo do storage no Serial.
- void deleteOneMessage ()

  Remove o primeiro registro do storage.
- bool hasDataStorage ()

  Verifica se existem registros armazenados.

4.29.1 Detailed Description

Interface pública do módulo de armazenamento persistente utilizando LittleFS.

Este módulo define as estruturas de dados e as funções responsáveis pelo gerenciamento do armazenamento local de dados ambientais (temperatura e umidade) no sistema de arquivos LittleFS do microcontrolador.

O objetivo principal é fornecer um mecanismo de persistência confiável, permitindo que os dados sejam armazenados localmente quando não for possível enviá-los ao servidor remoto (por exemplo, em caso de falha na comunicação MQTT), e posteriormente recuperados e retransmitidos.

Funcionalidades fornecidas:

- Inicialização do sistema de arquivos;
- Armazenamento persistente de dados em formato JSON;
- Recuperação de registros armazenados;
- Remoção sequencial de registros;
- Verificação de existência de dados pendentes;
- Impressão dos dados armazenados para depuração.

Estrutura de armazenamento:

- Os dados são armazenados em formato JSON;
- Cada linha do arquivo representa um registro independente;
- Cada registro contém:

```
{
  "t": 25.40,
  "h": 60.20
}
```

Fluxo típico de utilização:

```
initStorage();

saveStorage(25.3, 60.2);

ObjectStorage obj = getObjectStorage();

if (obj.valid) {
    // processar dados
    deleteOneMessage();
}
```

Aplicações típicas:

- Buffer local para sistemas IoT;
- Armazenamento temporário em caso de falha de rede;
- Sistemas com envio assíncrono de dados;
- Sistemas tolerantes a falhas.

Dependências:

- LittleFS
- ArduinoJson
- ESP8266 / ESP32 Arduino Core

Arquivo relacionado:

See also

storage.cpp

Warning

A função initStorage() deve ser chamada antes de qualquer operação.

Note

Este módulo implementa uma fila persistente FIFO simples.

4.29.2 Function Documentation

4.29.2.1 deleteOneMessage()

```
void deleteOneMessage ()
```

Remove o primeiro registro do storage.

- Copia todos os registros, exceto o primeiro, para um arquivo temporário.
- Substitui o arquivo original pelo temporário.

Remove o primeiro registro do storage.

- Cria um arquivo temporário.
- Copia todas as linhas do arquivo original, exceto a primeira.
- Substitui o arquivo original pelo temporário.

4.29.2.2 getObjectStorage()

```
ObjectStorage getObjectStorage (
            size_t index)
```

Obtém um registro do storage.

- Lê o arquivo linha por linha.
- Retorna o objeto localizado no índice especificado.

Parameters

index	Índice do objeto a recuperar (0 = primeiro elemento).

Returns

Estrutura ObjectStorage com os dados recuperados. Se o índice não for encontrado ou os dados forem inválidos, retorna um objeto com `valid = false`.

Obtém um registro do storage.

- Cada linha do arquivo representa um objeto JSON.
- A leitura é feita até alcançar o índice informado.
- Caso os dados sejam válidos, retorna um objeto preenchido.
- Caso contrário, retorna um objeto inválido (`valid = false`).

Parameters

index	Índice (linha) do objeto a ser recuperado.

Returns

Estrutura `ObjectStorage` contendo temperatura, umidade e validade.

4.29.2.3 hasDataStorage()

```
bool hasDataStorage ()
```

Verifica se existem registros armazenados.

Returns

true se há ao menos um registro no storage.

false caso contrário.

Verifica se existem registros armazenados.

Returns

true se existe pelo menos um objeto armazenado.

false caso contrário.

4.29.2.4 initStorage()

```
void initStorage ()
```

Inicializa o sistema de armazenamento.

- Monta o sistema de arquivos LittleFS.
- Essa funçao deve ser chamada no setup().

Inicializa o sistema de armazenamento.

- Monta a partição LittleFS.
- Caso a montagem falhe, gera um log de erro.

4.29.2.5 saveStorage()

```
void saveStorage (
            float temperature,
            float humidity)
```

Salva um novo registro de temperatura e umidade no storage.

Os dados são armazenados em formato JSON, cada registro em uma linha.

Parameters

temperature	Valor de temperatura a salvar.
humidity	Valor de umidade a salvar.

Salva um novo registro de temperatura e umidade no storage.

- Abre o arquivo em modo append (`"a"`).
- Cria um documento JSON com os valores fornecidos.
- Grava os dados no arquivo e fecha.
- Atualiza a flag `hasObj`.

Parameters

temperature	Valor da temperatura a ser armazenada.
humidity	Valor da umidade a ser armazenada.

4.29.2.6 storagePrint()

```
void storagePrint ()
```

Imprime todo o conteúdo do storage no Serial.

- Lê o arquivo linha por linha.
- Desserializa e exibe cada objeto armazenado em formato legível.

Imprime todo o conteúdo do storage no Serial.

- Lê o arquivo linha por linha.
- Desserializa cada linha em JSON.
- Exibe temperatura e umidade formatadas no console.

4.30 storage.h

Go to the documentation of this file.

```
#ifndef STORAGE_H
#define STORAGE_H

#include <Arduino.h>

// ---------------------------------------------------------------------------
// Estrutura de dados armazenada
// ---------------------------------------------------------------------------

struct ObjectStorage {
  float temperature = 0.0f;
  float humidity    = 0.0f;
  bool valid        = false;
};

// ---------------------------------------------------------------------------
// Funções da API de storage
// ---------------------------------------------------------------------------

void initStorage();

void saveStorage(float temperature, float humidity);

ObjectStorage getObjectStorage(size_t index = 0);

void storagePrint();

void deleteOneMessage();

bool hasDataStorage();

#endif // STORAGE_H
```

4.31 microcontrolador/wifi.cpp File Reference

Implementação do gerenciamento de conectividade Wi-Fi do dispositivo.

```
#include <ESP8266WiFi.h>
#include "wifi.h"
#include "log.h"
#include "config.h"
#include "config_storage.h"
```

Functions

- bool connectWiFi ()

  Conecta o dispositivo à rede Wi-Fi configurada.
- bool reconnectWifi ()

  Mantém a conexão Wi-Fi ativa, tentando reconectar quando necessário.
- void createAccessPoint ()

  Cria um ponto de acesso (Access Point) para configuração local.

Variables

- char ipBuffer [16]

  Buffer que armazena o endereço IP do dispositivo.

4.31.1 Detailed Description

Implementação do gerenciamento de conectividade Wi-Fi do dispositivo.

Este arquivo contém as funções responsáveis por controlar a conexão Wi-Fi do sistema embarcado, incluindo:

- Conexão à rede Wi-Fi configurada previamente.
- Reconexão automática em caso de perda de conexão.
- Criação de um Access Point (AP) para configuração local do dispositivo.
- Armazenamento e disponibilização do endereço IP atual.

O módulo utiliza as credenciais armazenadas na estrutura global de configuração (`cfg`), que são persistidas em memória através do subsistema de armazenamento.

Também implementa mecanismos de controle de tempo para evitar tentativas excessivas de reconexão, melhorando a estabilidade e reduzindo o consumo de energia.

Este arquivo depende dos seguintes módulos:

- wifi.h Interface pública do módulo Wi-Fi
- config.h Definições de configuração do sistema
- config_storage.h Persistência das configurações
- log.h Sistema de logging

Note

Este módulo foi desenvolvido para a plataforma ESP8266 utilizando a biblioteca ESP8266WiFi.

Date

2026

4.31.2 Function Documentation

4.31.2.1 connectWiFi()

```
bool connectWiFi ()
```

Conecta o dispositivo à rede Wi-Fi configurada.

Esta função utiliza o SSID e senha definidos em `cfg.wifiSSID` e `cfg.wifiPass`. São obtidos a partir da estrutura global `cfg`, que contém as variáveis persistidas. Ela tenta estabelecer uma conexão com a rede Wi-Fi e aguarda até 5 segundos para o processo ser concluído.

- Se a conexão for bem-sucedida, o endereço IP é obtido e registrado no log.
- Caso contrário, retorna `false` indicando falha na conexão.

Returns

true Se a conexão foi estabelecida com sucesso.

false Se não foi possível conectar dentro do tempo limite.

4.31.2.2 createAccessPoint()

```
void createAccessPoint ()
```

Cria um ponto de acesso (Access Point) para configuração local.

Caso o dispositivo não esteja em modo AP, esta função:

- Desconecta o Wi-Fi atual.
- Configura o modo `WIFI_AP`.
- Inicializa um Access Point com o SSID e senha definidos nas constantes `SSID_ACCESS_POINT` e `PASSWORD_ACCESS_POINT`.

Também imprime o endereço IP do AP via `Serial`.

4.31.2.3 reconnectWifi()

```
bool reconnectWifi ()
```

Mantém a conexão Wi-Fi ativa, tentando reconectar quando necessário.

Verifica o estado da conexão Wi-Fi e tenta reconectar quando necessário.

Esta função deve ser chamada periodicamente (ex: no loop principal).

- Se o dispositivo estiver conectado, nada é feito.
- Caso contrário, a cada 1 minuto e 15 segundos, uma nova tentativa de conexão é feita.
- Se uma tentativa for feita, a função chama internamente `connectWiFi()`.

Returns

true Se o dispositivo está ou foi reconectado com sucesso.

false Se ainda não está conectado e o tempo de reconexão não chegou.

4.31.3 Variable Documentation

4.31.3.1 ipBuffer

```
char ipBuffer[16]
```

Buffer que armazena o endereço IP do dispositivo.

Este buffer contém o endereço IP no formato string IPv4, por exemplo: "192.168.1.10".

O tamanho de 16 bytes é suficiente para armazenar o maior endereço IPv4 possível (15 caracteres) mais o caractere nulo de terminação '\0'.

4.32 wifi.h

```
00001
00010
00011 #ifndef WIFI_HELPER_H
00012 #define WIFI_HELPER_H
00013
00014
00024 bool connectWiFi();
00025
00035 bool reconnectWifi();
00036
00044 void createAccessPoint();
00045
00046 #endif
```

Chapter 5

Examples

5.1 C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/led.cpp

Alterna o estado do LED em intervalos definidos.

Alterna o estado do LED em intervalos definidos.Esta função implementa um temporizador não bloqueante utilizando millis(). O LED muda de estado somente quando o intervalo definido em LED_INTERVAL_MS é atingido.

Isso permite que o sistema continue executando outras tarefas simultaneamente.

Parameters

now	Tempo atual do sistema em milissegundos. Normalmente obtido através da função:

```
millis()
toggleLed(millis());
```

Note

Não utiliza delay(), garantindo melhor desempenho do sistema.

```
#include "led.h"
#include "config.h"

// ---------------------------------------------------------------------------
// Variáveis globais privadas ao módulo
// ---------------------------------------------------------------------------

unsigned long lastTimeLedTurnedOn = 0;

constexpr unsigned long LED_INTERVAL_MS = 2000;

// ---------------------------------------------------------------------------
// Implementação das funções públicas
// ---------------------------------------------------------------------------

void ledInit()
{
```

```
    pinMode(PIN_LED, OUTPUT);

    // Garante estado inicial desligado
    digitalWrite(PIN_LED, LOW);
}


void toggleLed(unsigned long now)
{
    if (now - lastTimeLedTurnedOn >= LED_INTERVAL_MS)
    {
        lastTimeLedTurnedOn = now;

        // Alterna estado atual
        digitalWrite(PIN_LED, !digitalRead(PIN_LED));
    }
}


void turnOnLed()
{
    digitalWrite(PIN_LED, HIGH);
}


void turnOffLed()
{
    digitalWrite(PIN_LED, LOW);
}
```

5.2 C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/led.h

Alterna o estado do LED em intervalos de tempo definidos.

Alterna o estado do LED em intervalos de tempo definidos.Implementa um mecanismo de temporização não bloqueante utilizando millis(). O LED alterna entre ligado e desligado automaticamente após o intervalo definido internamente no módulo.

Esta abordagem permite que o sistema continue executando outras tarefas simultaneamente.

Parameters

now	Tempo atual do sistema em milissegundos.

Deve ser obtido através da função:

```
millis()
toggleLed(millis());
```

Note

Não utiliza delay(), evitando bloqueio do sistema.

```
#ifndef LED_H
#define LED_H
#include <Arduino.h>


// ------------------------------------------------------------------------------
// Funções públicas do módulo
```

```
// ---------------------------------------------------------------------------

void ledInit();


void toggleLed(unsigned long now);


void turnOnLed();


void turnOffLed();


#endif
```

5.3 C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.cpp

Inicializa o cliente MQTT.

Inicializa o cliente MQTT.Configura:

- Endereço do broker;
- Porta de comunicação;

Esta função deve ser chamada durante o setup do sistema.

```
setupMQTT();
```

```
#include "config.h"
#include "mqtt.h"
#include "log.h"
#include <WiFiClientSecure.h>
#include <ArduinoJson.h>
#include "storage.h"
#include "config_storage.h"


// ---------------------------------------------------------------------------
// Objetos MQTT
// ---------------------------------------------------------------------------

WiFiClient espClient;


PubSubClient client(espClient);


// ---------------------------------------------------------------------------
// Variáveis de controle de reconexão
// ---------------------------------------------------------------------------

static unsigned long lastAttempConnectMQTT = 0;


static const unsigned long reconnectIntervalMQTT = 3000;


static unsigned long lastConnectionDataSent = 0;


static const unsigned long resendIntervalMQTT = 3000;


bool wasSendAlert = false;


// ---------------------------------------------------------------------------
```

```
// Inicialização
// ---------------------------------------------------------------------------

void setupMQTT()
{
    client.setServer(cfg.mqttServer.c_str(), cfg.mqttPort);

    log(LOG_DEBUG, "MQTT inicializado");
}


void resetMQTTClient()
{
    log(LOG_WARN, "Reinicializando cliente MQTT e TLS...");

    espClient = WiFiClient();

    client.setServer(cfg.mqttServer.c_str(), cfg.mqttPort);

    client.setClient(espClient);

    log(LOG_DEBUG, "Cliente MQTT reconfigurado");
}


// ---------------------------------------------------------------------------
// Gerenciamento de conexão
// ---------------------------------------------------------------------------

bool checkMQTTConnected()
{
    if (client.connected())
    {
        client.loop();

        return true;
    }

    unsigned long now = millis();

    if (now - lastAttempConnectMQTT >= reconnectIntervalMQTT)
    {
        lastAttempConnectMQTT = now;

        log(LOG_DEBUG, "Tentando conectar ao MQTT...");

        if (client.connect(
                cfg.mqttDeviceId.c_str(),
                cfg.mqttUser.c_str(),
                cfg.mqttPass.c_str()))
        {
            log(LOG_INFO, "Conectado ao broker MQTT");

            return true;
        }
        else
        {
            log(LOG_ERROR, "Erro ao conectar com broker (rc=%d)", client.state());

            static uint8_t connection_drop_count = 0;

            connection_drop_count++;

            if (connection_drop_count >= 3)
            {
                resetMQTTClient();

                connection_drop_count = 0;
            }

            return false;
        }
    }

    return false;
}


// ---------------------------------------------------------------------------
// Publicação de dados
// ---------------------------------------------------------------------------

bool publishSensorData(float temperature, float humidity)
{
```

```
    StaticJsonDocument<128> doc;

    doc["microcontrollerId"] = cfg.mqttDeviceId.c_str();

    doc["temperature"] = temperature;

    doc["humidity"] = humidity;

    char buffer[128];

    serializeJson(doc, buffer);

    if (!client.publish(cfg.mqttTopicData.c_str(), buffer))
    {
        log(LOG_WARN, "Falha ao enviar dados ao broker MQTT");

        return false;
    }

    return true;
}


// ---------------------------------------------------------------------------
// Reenvio de dados armazenados
// ---------------------------------------------------------------------------

void resendMqttData()
{
    if (!hasDataStorage())
        return;

    unsigned long now = millis();

    if (now - lastConnectionDataSent >= resendIntervalMQTT)
    {
        lastConnectionDataSent = now;

        ObjectStorage obj = getObjectStorage();

        if (obj.valid)
        {
            if (publishSensorData(obj.temperature, obj.humidity))
            {
                deleteOneMessage();
            }
        }
    }
}


// ---------------------------------------------------------------------------
// Publicação de alertas
// ---------------------------------------------------------------------------

void publishAlert(float temperature, float humidity)
{
    if (!wasSendAlert)
    {
        StaticJsonDocument<128> doc;

        doc["microcontrollerId"] = cfg.mqttDeviceId.c_str();

        doc["temperature"] = temperature;

        doc["humidity"] = humidity;

        char buffer[128];

        serializeJson(doc, buffer);

        client.publish(cfg.mqttTopicAlert.c_str(), buffer);

        wasSendAlert = true;

        log(LOG_INFO, "alerta publicado no mqtt");
    }
}


void resetStateSendAlert()
{
    wasSendAlert = false;
}
```

5.4 C:/Users/fabic/Documents/projetos_Embarcatech_residencia/projetos/Projetos/monitoramento-de-temperatura/microcontrolador/mqtt.h

Inicializa o cliente MQTT.

Inicializa o cliente MQTT.Configura:

- Endereço do broker;
- Porta de comunicação;
- Cliente de rede associado.

Deve ser chamada uma vez durante a inicialização do sistema.

```
void setup()
{
    setupMQTT();
}
```

```
#ifndef MQTT_HELPER_H
#define MQTT_HELPER_H

#include <PubSubClient.h>


// ---------------------------------------------------------------------------
// Variáveis globais
// ---------------------------------------------------------------------------

extern PubSubClient client;


// ---------------------------------------------------------------------------
// Funções públicas
// ---------------------------------------------------------------------------

void setupMQTT();


bool checkMQTTConnected();


bool publishSensorData(float temperature, float humidity);


void resendMqttData();


void publishAlert(float temperature, float humidity);


void resetStateSendAlert();


#endif
```

Index

aht
- sensor.cpp, 64
- sensor.h, 66

button.cpp
- buttonInit, 11
- buttonPressStartTime, 15
- checkButtonPressed, 11
- disableButtonInterruptChange, 11
- disableButtonInterruptRising, 12
- enableButtonInterruptChange, 12
- enableButtonInterruptRising, 12
- flagLongPress, 15
- flagMutePressed, 15
- flagShortPress, 15
- isButtonPressed, 15
- isMuted, 16
- longPressButtonISR, 13
- muteButtonISR, 13
- resetButtonState, 13
- wasButtonLongPressed, 13
- wasButtonShortPress, 14
- wasMuted, 14

button.h
- buttonInit, 17
- disableButtonInterruptChange, 17
- disableButtonInterruptRising, 18
- enableButtonInterruptChange, 18
- enableButtonInterruptRising, 19
- resetButtonState, 19
- wasButtonLongPressed, 20
- wasButtonShortPress, 20
- wasMuted, 21

buttonInit
- button.cpp, 11
- button.h, 17

buttonPressStartTime
- button.cpp, 15

buzzer.cpp
- buzzerInit, 23
- disableSoundAlert, 23
- lastBuzzerPlayTime, 24
- toggleBuzzer, 23

buzzer.h
- BUZZER_INTERVAL_MS, 26
- buzzerInit, 25
- disableSoundAlert, 25
- toggleBuzzer, 25

BUZZER_INTERVAL_MS
- buzzer.h, 26

buzzerInit
- buzzer.cpp, 23
- buzzer.h, 25

cfg
- config_storage.cpp, 33
- config_storage.h, 39

checkButtonPressed
- button.cpp, 11

clearLine
- display_LCD-1602_I2C.cpp, 42

client
- mqtt.cpp, 54
- mqtt.h, 61

config.h
- PASSWORD_ACCESS_POINT, 28
- PIN_BUTTON_LONG, 28
- PIN_BUTTON_MUTE, 28
- PIN_BUZZER, 28
- PIN_LED, 28
- SSID_ACCESS_POINT, 29

CONFIG_PATH
- config_storage.cpp, 33

config_storage.cpp
- cfg, 33
- CONFIG_PATH, 33
- hasConfigData, 31
- initConfigStorage, 31
- loadConfig, 31
- saveConfigutionData, 32

config_storage.h
- cfg, 39
- hasConfigData, 35
- initConfigStorage, 36
- loadConfig, 36
- saveConfigutionData, 37

ConfigurationData, 5
- humidityMin, 6
- mqttPort, 6
- mqttServer, 6
- valid, 6
- wifiPass, 7

connectWiFi
- wifi.cpp, 81

createAccessPoint
- wifi.cpp, 81

CURRENT_LOG_LEVEL
- log.cpp, 50
- log.h, 52

deleteOneMessage
storage.cpp, 72
storage.h, 77
disableButtonInterruptChange
button.cpp, 11
button.h, 17
disableButtonInterruptRising
button.cpp, 12
button.h, 18
disableSoundAlert
buzzer.cpp, 23
buzzer.h, 25
display_LCD-1602_I2C.cpp
clearLine, 42
handleBacklightLCD, 42
lcd1602_init, 42
lcd1602_showData, 42
show_value, 43
TEMP_UPDATE_THRESHOLD, 43

enableButtonInterruptChange
button.cpp, 12
button.h, 18
enableButtonInterruptRising
button.cpp, 12
button.h, 19
EnvironmentData, 7
espClient
mqtt.cpp, 57

flagLongPress
button.cpp, 15
flagMutePressed
button.cpp, 15
flagShortPress
button.cpp, 15

getObjectStorage
storage.cpp, 72
storage.h, 77

handleBacklightLCD
display_LCD-1602_I2C.cpp, 42
hasConfigData
config_storage.cpp, 31
config_storage.h, 35
hasDataStorage
storage.cpp, 73
storage.h, 78
humidityMin
ConfigurationData, 6

initConfigStorage
config_storage.cpp, 31
config_storage.h, 36
initializeSensor
sensor.cpp, 64
sensor.h, 65
initStorage
storage.cpp, 73
storage.h, 78
ipBuffer
wifi.cpp, 82
isButtonPressed
button.cpp, 15
isMuted
button.cpp, 16

lastBuzzerPlayTime
buzzer.cpp, 24
lastTimeLedTurnedOn
led.cpp, 46
lcd1602_init
display_LCD-1602_I2C.cpp, 42
lcd1602_showData
display_LCD-1602_I2C.cpp, 42
led.cpp
lastTimeLedTurnedOn, 46
LED_INTERVAL_MS, 46
ledInit, 45
turnOffLed, 45
turnOnLed, 45
led.h
ledInit, 47
turnOffLed, 47
turnOnLed, 48
LED_INTERVAL_MS
led.cpp, 46
ledInit
led.cpp, 45
led.h, 47
loadConfig
config_storage.cpp, 31
config_storage.h, 36
log
log.cpp, 49
log.h, 51
log.cpp
CURRENT_LOG_LEVEL, 50
log, 49
logInit, 49
log.h
CURRENT_LOG_LEVEL, 52
log, 51
LOG_DEBUG, 51
LOG_ERROR, 51
LOG_INFO, 51
LOG_NONE, 51
LOG_WARN, 51
logInit, 52
LogLevel, 51
LOG_DEBUG
log.h, 51
LOG_ERROR
log.h, 51
LOG_INFO
log.h, 51
LOG_NONE

log.h, 51
LOG_WARN
log.h, 51
logInit
log.cpp, 49
log.h, 52
LogLevel
log.h, 51
longPressButtonISR
button.cpp, 13

microcontrolador/button.cpp, 9
microcontrolador/button.h, 16, 22
microcontrolador/buzzer.cpp, 22
microcontrolador/buzzer.h, 24, 26
microcontrolador/config.h, 27, 29
microcontrolador/config_storage.cpp, 30
microcontrolador/config_storage.h, 34, 39
microcontrolador/display_LCD-1602_I2C.cpp, 40
microcontrolador/display_LCD-1602_I2C.h, 44
microcontrolador/led.cpp, 44
microcontrolador/led.h, 46, 48
microcontrolador/log.cpp, 48
microcontrolador/log.h, 50, 53
microcontrolador/mqtt.cpp, 53
microcontrolador/mqtt.h, 57, 62
microcontrolador/sensor.cpp, 62
microcontrolador/sensor.h, 65, 66
microcontrolador/server.h, 67, 70
microcontrolador/storage.cpp, 70
microcontrolador/storage.h, 74, 80
microcontrolador/wifi.cpp, 80
microcontrolador/wifi.h, 82
mqtt.cpp
client, 54
espClient, 57
publishAlert, 54
publishSensorData, 55
resendMqttData, 55
resetMQTTClient, 56
resetStateSendAlert, 56
wasSendAlert, 57
mqtt.h
client, 61
publishAlert, 59
publishSensorData, 59
resendMqttData, 60
resetStateSendAlert, 61
mqttPort
ConfigurationData, 6
mqttServer
ConfigurationData, 6
muteButtonISR
button.cpp, 13

ObjectStorage, 8

PASSWORD_ACCESS_POINT
config.h, 28
PIN_BUTTON_LONG
config.h, 28
PIN_BUTTON_MUTE
config.h, 28
PIN_BUZZER
config.h, 28
PIN_LED
config.h, 28
publishAlert
mqtt.cpp, 54
mqtt.h, 59
publishSensorData
mqtt.cpp, 55
mqtt.h, 59

readSensorData
sensor.cpp, 64
sensor.h, 65
reconnectWifi
wifi.cpp, 82
resendMqttData
mqtt.cpp, 55
mqtt.h, 60
resetButtonState
button.cpp, 13
button.h, 19
resetMQTTClient
mqtt.cpp, 56
resetStateSendAlert
mqtt.cpp, 56
mqtt.h, 61

saveConfigutionData
config_storage.cpp, 32
config_storage.h, 37
saveStorage
storage.cpp, 73
storage.h, 78
sensor.cpp
aht, 64
initializeSensor, 64
readSensorData, 64
sensor.h
aht, 66
initializeSensor, 65
readSensorData, 65
server.h
server_handle_loop, 68
server_setup_routes, 68
startServer, 69
stopServer, 69
server_handle_loop
server.h, 68
server_setup_routes
server.h, 68
show_value
display_LCD-1602_I2C.cpp, 43
SSID_ACCESS_POINT
config.h, 29

startServer
server.h, 69
stopServer
server.h, 69
storage.cpp
deleteOneMessage, 72
getObjectStorage, 72
hasDataStorage, 73
initStorage, 73
saveStorage, 73
storagePrint, 74
storage.h
deleteOneMessage, 77
getObjectStorage, 77
hasDataStorage, 78
initStorage, 78
saveStorage, 78
storagePrint, 79
storagePrint
storage.cpp, 74
storage.h, 79

TEMP_UPDATE_THRESHOLD
display_LCD-1602_I2C.cpp, 43
toggleBuzzer
buzzer.cpp, 23
buzzer.h, 25
turnOffLed
led.cpp, 45
led.h, 47
turnOnLed
led.cpp, 45
led.h, 48

valid
ConfigurationData, 6

wasButtonLongPressed
button.cpp, 13
button.h, 20
wasButtonShortPress
button.cpp, 14
button.h, 20
wasMuted
button.cpp, 14
button.h, 21
wasSendAlert
mqtt.cpp, 57
wifi.cpp
connectWiFi, 81
createAccessPoint, 81
ipBuffer, 82
reconnectWifi, 82
wifiPass
ConfigurationData, 7